

A HANDS-ON JAVASCRIPT COURSE

Design Patterns

All 23 classic (Gang of Four) design patterns — the naive problem, the pattern that fixes it, and an exercise for each.

Compiled for reading — run the code from the original project to experiment.

Design Patterns Course (JavaScript) — run locally

Har pattern ko **chala kar** samajhne ke liye. Koi npm install ki zaroorat nahi — sab kuch pure Node hai.

Requirement

- Node.js 18+ installed. Check karo:

```
node --version
```

Agar nahi hai: <https://nodejs.org> se LTS install karo.

Kaise use karenin (har lesson ke liye same flow)

1. Lesson folder ka `README.md` padho.
2. `1-naive.js` chalao — dekho problem kaisi dikhti hai:

```
node 01-strategy/1-naive.js
```

1. Pattern wali files chalao — dekho kaise clean hota hai.
2. `exercise.js` khol ke khud solve karo. Chala kar test karo.

Structure — saare 23 classic (GoF) patterns

Creational — object *kaise banta hai*

```
design-patterns-course/  
├─ 04-singleton/      ← sirf ek instance guarantee karo  
├─ 05-factory-method/ ← object banane ka kaam ek function ko do  
├─ 06-abstract-factory/ ← related objects ka poora consistent family banao  
├─ 07-builder/        ← complex object ko step-by-step banao  
├─ 08-prototype/      ← naye instance existing object ko CLONE karke banao
```

Structural — objects *kaise jude/compose hote hain*

— 09-adapter/	← mismatched interface ko apne shape mein convert karo
— 10-bridge/	← do independent dimensions ko alag hierarchies mein todo
— 11-composite/	← tree (leaf + branch) ko uniform interface do
— 12-decorator/	← runtime pe stackable behavior jodo
— 13-facade/	← complex subsystem ke aage ek simple entry point
— 14-flyweight/	← shared data reuse karo, memory bachao
— 15-proxy/	← real object ke aage access-control/lazy-load stand-in

Behavioral — objects *kaise interact/communicate karte hain*

— 01-strategy/ shuru karo)	← runtime pe interchangeable algorithms (yahin se shuru karo)
— 02-observer/ chale	← ek-se-anek: state badle, sab subscribers ko pata chale
— 03-command/	← action ko object banao (undo/queue/replay)
— 16-chain-of-responsibility/	← handlers ki chain, jo handle kar sake wahi kare
— 17-iterator/ karo	← kisi bhi collection ko uniform tarike se traverse karo
— 18-mediator/	← components sirf ek central mediator se baat karein
— 19-memento/	← object khud apna snapshot save/restore kare
— 20-state/ transition kare	← behavior current status pe depend kare, state khud transition kare
— 21-template-method/	← shared skeleton, sirf steps subclass mein override
— 22-visitor/	← classes ko chhue bina naye operations add karo
— 23-interpreter/	← chhoti grammar ko composable nodes se evaluate karo
— package.json	← shortcut scripts (optional)

Shortcut scripts (optional)

`package.json` mein scripts hain, toh yeh bhi kaam karta hai:

```
npm run strategy:registry
npm run observer:emitter
npm run singleton:class
npm run visitor:pattern
```

Poori list ke liye `package.json` kholo — har lesson ke `naive` /pattern

files ke liye ek script hai.

Golden rule

Sirf padhna nahi — **chalao, tood-fod karo, dobara chalao.** Jo pattern tumne

khud break karke wapas jode ho, wahi yaad rehta hai.

Table of Contents

CREATIONAL PATTERNS

1. Singleton
2. Factory Method
3. Abstract Factory
4. Builder
5. Prototype

STRUCTURAL PATTERNS

6. Adapter
7. Bridge
8. Composite
9. Decorator
10. Facade
11. Flyweight
12. Proxy

BEHAVIORAL PATTERNS

13. Strategy
14. Observer
15. Command
16. Chain of Responsibility
17. Iterator
18. Mediator
19. Memento
20. State
21. Template Method
22. Visitor
23. Interpreter

1. Singleton

Problem: kuch cheezein app mein **sirf ek hi** honi chahiye — config, connection pool, logger. Agar har jagah `new` karte raho to alag-alag state ban jaata hai aur sab out of sync ho jaate hain.

Idea: class khud control kare ki uska sirf ek instance bane. Pehli baar `new` / `getInstance()` call hone par instance banta hai, uske baad wahi wapas milta hai.

Files — is order mein chalao

```
node 1-naive.js          # ❌ har `new Config()` alag object
node 2-singleton.js      # ✅ class + static instance
node 3-singleton-module.js # ✅ idiomatic JS: module scope khud singleton hai
node exercise.js         # 🔄 tumhara turn
```

Principles yahan dikh rahe hain

1. **Single source of truth** — ek hi state, jahan se bhi access karo.
2. **Controlled instantiation** — constructor khud check karta hai ki instance already ban chuka hai ya nahi.

When to skip

Singleton **global mutable state** hai — testing mushkil kar deta hai (state tests ke beech leak hota hai) aur hidden dependencies bana deta hai. Agar dependency injection se kaam chal sakta hai (function/constructor ko object pass karo), woh usually behtar hai. JS mein simple config/constants ke liye module export (`3-singleton-module.js`) aksar kaafi hai — poora class/pattern zaroori nahi.

❌ 1-naive.js — the naive approach

```
// ❌ THE NAIVE VERSION — har jagah naya config object bana rahe hain.
// Chalao: node 1-naive.js
```

```

class Config {
  constructor() {
    this.settings = { theme: 'light', retries: 3 };
  }
}

const configA = new Config();
const configB = new Config();

configA.settings.theme = 'dark';
console.log('configA.theme:', configA.settings.theme);
console.log('configB.theme:', configB.settings.theme); // still 'light' - inconsistent!

// PROBLEM: Config app-wide ek hi hona chahiye (ek hi source of truth). Har
// `new Config()` alag state banata hai - kahin change karo, kahin nahi dikhta.

```

✓ 2-singleton.js — the pattern

```

// ✓ SINGLETON PATTERN - sirf ek instance, jahan se bhi maango wahi milega.
// Chalao: node 2-singleton.js

class Config {
  static #instance;

  constructor() {
    if (Config.#instance) return Config.#instance; // already bana hai to wahi do
    this.settings = { theme: 'light', retries: 3 };
    Config.#instance = this;
  }

  static getInstance() {
    return Config.#instance ?? new Config();
  }
}

const configA = Config.getInstance();
const configB = Config.getInstance();

configA.settings.theme = 'dark';
console.log('configA.theme:', configA.settings.theme);
console.log('configB.theme:', configB.settings.theme); // 'dark' - same object hai
console.log('same instance?', configA === configB);

```

✓ 3-singleton-module.js — the pattern

```

// ✓ IDIOMATIC JS: ES module khud hi singleton hai - module-level object
// export karo, JS engine use sirf EK baar evaluate karta hai (import cache).
// Chalao: node 3-singleton-module.js

const settings = { theme: 'light', retries: 3 }; // module scope = automatically singleton
export default settings;

```

```
// Isko do jagah "import" karke dikhate hain (simulate):
const configA = settings;
const configB = settings;
configA.theme = 'dark';
console.log('configA.theme:', configA.theme);
console.log('configB.theme:', configB.theme); // 'dark' – same reference

// Yehi tarika JS mein sabse zyada use hota hai: class + getInstance() ki
// zaroorat kam padti hai, module system khud singleton de deta hai.
```

exercise.js — your turn

```
// 🐛 EXERCISE – Logger ko Singleton banao taaki poore app mein ek hi log
// history ho.
// Chalao: node exercise.js

// ----- naive version (isse refactor karna hai) -----
class Logger {
  constructor() { this.logs = []; }
  log(msg) { this.logs.push(msg); console.log('LOG:', msg); }
}

const loggerA = new Logger();
const loggerB = new Logger();
loggerA.log('user signed up');
loggerB.log('payment received');
console.log('loggerA has', loggerA.logs.length, 'logs'); // 1 – but chahiye 2!

// =====
// TODO 1: Logger class mein static #instance field + getInstance() add karo
//         (2-singleton.js jaisa), taaki loggerA aur loggerB same object hon.
//
// TODO 2 (bonus): ek module-level `export default new Logger()` variant bhi
//                 try karo (3-singleton-module.js jaisa) – dono approach ka tradeoff
//                 socho: class singleton lazy hai (pehli baar use hone par banta
//                 hai), module singleton eager hai (import hote hi ban jaata hai).
//
// Neeche apna solution likho, phir `node exercise.js` chala kar verify karo.
// =====
```

2. Factory Method

Problem: object banane ka logic (`new X()` , kaunsi class chuni) caller ke code mein bikhra hua hai. Naya type add karna matlab har jagah if/else dhoondh ke edit karna.

Idea: object banane ka kaam ek dedicated factory function/method ko do.

Caller sirf "mujhe X type chahiye" bolta hai, konsi class instantiate hui — uski chinta nahi karta.

Files — is order mein chalao

node 1-naive.js	# ❌ caller khud class chunta hai
node 2-factory-method.js	# ✅ factory function + registry
node exercise.js	# 🎯 tumhara turn

Principles yahan dikh rahe hain

1. **Separate creation from use** — "kya banaya" aur "kaise use kiya" alag.
2. **Open/closed** — naya type = registry mein entry, caller code same rehta.

When to skip

Sirf ek hi class hai jo kabhi variant nahi legi? Seedha `new X()` karo — factory ki indirection tabhi kamaati hai jab **multiple related types** ho jinke beech runtime pe chunna ho.

❌ 1-naive.js — the naive approach

```
// ❌ THE NAIVE VERSION — caller khud decide karta hai KAUNSI class banani hai.
// Chalao: node 1-naive.js

class EmailNotification {
  send(msg) { console.log(`📧 Email: ${msg}`); }
}

class SmsNotification {
  send(msg) { console.log(`📱 SMS: ${msg}`); }
}

function notify(type, msg) {
```

```

let notification;
if (type === 'email') notification = new EmailNotification();
else if (type === 'sms') notification = new SmsNotification();
else throw new Error(`Unknown type: ${type}`);
notification.send(msg);
}

notify('email', 'Order shipped');
notify('sms', 'Order shipped');

// PROBLEM: naya notification type (push) add karne ke liye notify() function
// dobara edit karna padega. Creation Logic aur usage Logic mixed hain.

```

✓ 2-factory-method.js — the pattern

```

// ✓ FACTORY METHOD – object banane ka kaam ek dedicated function/method ko
// do, caller sirf "type" maangta hai, "how to build" nahi jaanta.
// Chalao: node 2-factory-method.js

class EmailNotification {
  send(msg) { console.log(`✉ Email: ${msg}`); }
}
class SmsNotification {
  send(msg) { console.log(`📱 SMS: ${msg}`); }
}
class PushNotification {
  send(msg) { console.log(`🔔 Push: ${msg}`); }
}

// Factory – ye "kaunsi class banani hai" ka decision yahin rakhta hai
function createNotification(type) {
  const notifications = {
    email: EmailNotification,
    sms: SmsNotification,
    push: PushNotification,
  };
  const NotificationClass = notifications[type];
  if (!NotificationClass) throw new Error(`Unknown type: ${type}`);
  return new NotificationClass();
}

function notify(type, msg) {
  const notification = createNotification(type); // caller ko class ka naam pata hi nahi
  notification.send(msg);
}

notify('email', 'Order shipped');
notify('push', 'Order shipped');

// Naya type add karna = registry mein ek line, notify() ko CHHUE BINA.

```

```
// 🐛 EXERCISE – shape banane ka if/else ko Factory Method mein refactor karo.
// Chalao: node exercise.js

// ----- naive version (isse refactor karna hai) -----
function makeShape(type, size) {
  if (type === 'circle') return { type, area: () => Math.PI * size * size };
  if (type === 'square') return { type, area: () => size * size };
  throw new Error(`Unknown shape: ${type}`);
}

console.log('circle area:', makeShape('circle', 3).area().toFixed(2));
console.log('square area:', makeShape('square', 3).area());

// =====
// TODO 1: `shapeFactories` registry object banao – { circle: (size) => {...},
//         square: (size) => {...} } – jismein har entry ek factory function
//         hai jo shape object return karta hai.
//
// TODO 2: `makeShape(type, size)` ko registry lookup karne wala banao (jaisa
//         2-factory-method.js mein hai).
//
// TODO 3 (bonus): 'triangle' add karo (area = base*height/2, size ko base aur
//         height dono maan lo ya function signature adjust karo) – makeShape
//         ko chhue bina.
//
// Neeche apna solution likho, phir `node exercise.js` chala kar verify karo.
// =====
```

3. Abstract Factory

Problem: kuch objects **families** mein aate hain jinka saath mein consistent rehna zaroori hai (theme ke button+checkbox, DB ke connection+query-builder). Agar har object alag jagah se banta hai, mismatch (dark button + light checkbox) possible ho jaata hai.

Idea: ek factory poori family banaye. Caller ek hi factory pakadta hai aur usi se saare related objects nikalta hai — mismatch structurally impossible ho jaata hai.

Files — is order mein chalao

```
node 1-naive.js           # ❌ har object alag factory function se — mismatch possible
node 2-abstract-factory.js # ✅ ek factory = poori consistent family
node exercise.js          # 🔄 tumhara turn
```

Principles yahan dikh rahe hain

1. **Factory Method ka bada bhai** — ek object ke bajaye, related objects ka poora *set* banata hai.

1. **Consistency by construction** — galat combination banana structurally mushkil ho jaata hai, runtime check ki zaroorat nahi.

When to skip

Agar objects ke beech koi "family" relationship nahi hai (wo independently change ho sakte hain), plain Factory Method (Lesson 5) kaafi hai — Abstract Factory ka extra layer overkill hoga.

❌ 1-naive.js — the naive approach

```
// ❌ THE NAIVE VERSION — button aur checkbox alag-alag banaye ja rahe hain,
// theme consistency guarantee nahi hoti.
// Chalao: node 1-naive.js

function createButton(theme) {
  return theme === 'dark'
```

```

    ? { render: () => console.log('🕒 [dark button]') }
    : { render: () => console.log('🕒 [light button]') };
  }
}

function createCheckbox(theme) {
  return theme === 'dark'
    ? { render: () => console.log('✅ [dark checkbox]') }
    : { render: () => console.log('❌ [light checkbox]') };
}

// Bug: kisi ne galti se mismatched theme pass kar diya
const button = createButton('dark');
const checkbox = createCheckbox('light'); // 😬 mismatch - koi compile-time check nahi
button.render();
checkbox.render();

// PROBLEM: button aur checkbox alag-alag jagah se banaye jaate hain - kuch bhi
// unhe "same family" (same theme) rehne ki guarantee nahi deta.

```

✅ 2-abstract-factory.js — the pattern

```

// ✅ ABSTRACT FACTORY - related objects ka poora "family" ek hi factory se
// banta hai, taaki wo hamesha consistent rahein (e.g. sab dark, ya sab light).
// Chalao: node 2-abstract-factory.js

const lightFactory = {
  createButton: () => ({ render: () => console.log('🕒 [light button]') }),
  createCheckbox: () => ({ render: () => console.log('❌ [light checkbox]') }),
};

const darkFactory = {
  createButton: () => ({ render: () => console.log('🕒 [dark button]') }),
  createCheckbox: () => ({ render: () => console.log('✅ [dark checkbox]') }),
};

function getFactory(theme) {
  const factories = { light: lightFactory, dark: darkFactory };
  return factories[theme] ?? (() => { throw new Error(`Unknown theme: ${theme}`); })();
}

// UI code ko sirf EK factory milta hai - mismatch ab possible hi nahi
function renderForm(factory) {
  factory.createButton().render();
  factory.createCheckbox().render();
}

console.log('-- dark theme form --');
renderForm(getFactory('dark'));
console.log('-- light theme form --');
renderForm(getFactory('light'));

// Naya theme (e.g. "high-contrast") add karna = naya factory object,
// renderForm() ko CHHUE BINA.

```

exercise.js — your turn

```
// 🐛 EXERCISE – postgres aur mysql "families" (connection + query builder)
// ko Abstract Factory se consistent banao.
// Chalao: node exercise.js

// ----- naive version (isse refactor karna hai) -----
function createConnection(db) {
  return db === 'mysql' ? { name: 'MySQL conn' } : { name: 'Postgres conn' };
}
function createQueryBuilder(db) {
  return db === 'mysql' ? { name: 'MySQL builder' } : { name: 'Postgres builder' };
}

const conn = createConnection('mysql');
const builder = createQueryBuilder('postgres'); // 😬 mismatch possible
console.log(conn.name, '+', builder.name);

// =====
// TODO 1: `mysqlFactory` aur `postgresFactory` objects banao, dono mein
//         `createConnection()` aur `createQueryBuilder()` methods hon.
//
// TODO 2: `getFactory(db)` function banao jo sahi factory return kare.
//
// TODO 3: `setup(factory)` function likho jo EK factory leke connection +
//         builder dono banaye – ab mismatch structurally impossible hona
//         chahiye.
//
// Neeche apna solution likho, phir `node exercise.js` chala kar verify karo.
// =====
```

4. Builder

Problem: ek object banane ke liye bohot saare (zyadatar optional) params chahiye. Constructor mein sabko positional pass karna unreadable hai aur order galat hone par silent bug ban jaata hai.

Idea: object ko ek baar mein nahi, **step-by-step methods** se banao. Har method ek field set karke `this` return kare (chaining), aakhir mein `build()` final object de.

Files — is order mein chalao

```
node 1-naive.js      # ❌ 6 positional params, order-sensitive
node 2-builder.js    # ✅ fluent chain, sirf jo chahiye set karo
node exercise.js     # 🎯 tumhara turn
```

Principles yahan dikh rahe hain

1. **Step-by-step construction** — complex object ek chain of readable calls se banta hai.

1. **Sensible defaults** — sirf jo customize karna hai woh set karo, baaki default rahe.

When to skip

Agar object mein 2-3 hi (mostly required) fields hain, plain object literal

`{ ... }` ya normal constructor kaafi hai — Builder tabhi kamaata hai jab fields zyada hon aur zyada optional hon.

❌ 1-naive.js — the naive approach

```
// ❌ THE NAIVE VERSION — constructor mein bohot saare positional params,
// order yaad rakhna mushkil, optional cheezon ke liye undefined pass karna
// padta hai.
// Chalao: node 1-naive.js

class HttpRequest {
  constructor(method, url, headers, body, timeout, retries) {
```

```

    this.method = method; this.url = url; this.headers = headers;
    this.body = body; this.timeout = timeout; this.retries = retries;
  }
}

// Headers ya body nahi chahiye? undefined pass karna padega, order galat
// hui to bug – pata bhi nahi chalega (retries ki jagah timeout chala jaayega)
const req = new HttpRequest('POST', '/api/orders', undefined, { id: 1 }, 5000, undefined);
console.log(req);

// PROBLEM: 6 positional args yaad rakhna, optional wale skip karne ke liye
// undefined chain karna, aur order-mistake ka silently wrong behavior.

```

✓ 2-builder.js — the pattern

```

// ✓ BUILDER PATTERN – object ko step-by-step, readable chaining se banao.
// Chalao: node 2-builder.js

class HttpRequest {
  constructor() {
    this.method = 'GET'; this.headers = {}; this.retries = 0;
  }
}

class HttpRequestBuilder {
  #request = new HttpRequest();

  setMethod(method) { this.#request.method = method; return this; }
  setUrl(url) { this.#request.url = url; return this; }
  setHeader(key, val) { this.#request.headers[key] = val; return this; }
  setBody(body) { this.#request.body = body; return this; }
  setTimeout(ms) { this.#request.timeout = ms; return this; }
  setRetries(n) { this.#request.retries = n; return this; }
  build() { return this.#request; }
}

// Fluent chain – sirf jo chahiye woh set karo, order matter nahi karta
const req = new HttpRequestBuilder()
  .setMethod('POST')
  .setUrl('/api/orders')
  .setHeader('Content-Type', 'application/json')
  .setBody({ id: 1 })
  .setTimeout(5000)
  .build();

console.log(req);

// Naya optional field add karna = ek naya `set...()` method, purane
// callers ko CHHUE BINA (woh method call hi nahi karenge to default rahega).

```

exercise.js — your turn

```
// 🐛 EXERCISE – pizza order ko Builder pattern se refactor karo.
// Chalao: node exercise.js

// ----- naive version (isse refactor karna hai) -----
function makePizza(size, crust, toppings, extraCheese) {
  return { size, crust, toppings: toppings ?? [], extraCheese: !!extraCheese };
}
console.log(makePizza('large', 'thin', ['mushroom', 'olives'], true));

// =====
// TODO 1: `PizzaBuilder` class banao with methods: setSize(size),
//          setCrust(crust), addTopping(name), setExtraCheese(bool), build().
//          Har set/add method `this` return kare (chaining ke liye).
//
// TODO 2: default size 'medium', default crust 'regular' rakho constructor
//          mein, toppings [] se start karo.
//
// TODO 3 (bonus): agar `build()` call hone tak size set nahi hua (still
//                  default) to warn console.log karo – "using default size".
//
// Neeche apna solution likho, phir `node exercise.js` chala kar verify karo.
// new PizzaBuilder().setSize('large').addTopping('mushroom').build()
// jaisa use hona chahiye.
// =====
```

5. Prototype

Problem: naya object banana expensive hai (heavy setup, nested config), lekin zyadatar naye objects kisi existing object se **milte-julte** hote hain — sirf 1-2 field alag.

Idea: ek "template" (prototype) object rakho. Naya instance scratch se banane ke bajaye prototype ko **clone** karo, phir jo chahiye woh change karo.

Files — is order mein chalao

```
node 1-naive.js      # ❌ har baar scratch se setup
node 2-prototype.js  # ✅ clone() se template reuse
node exercise.js     # 🔄 tumhara turn
```

Principles yahan dikh rahe hain

1. **Clone instead of construct** — setup cost ek hi baar, uske baad copy.
2. **Deep clone matters** — `structuredClone` (ya deep clone) use karo, warna nested objects/arrays accidentally share ho jaate hain (shallow `{...obj}` sirf top-level copy karta hai).

When to skip

Agar object banana already cheap hai (simple literal, no heavy setup), plain `{ ... }` ya factory function likhna simpler hai — cloning ki extra indirection ki zaroorat nahi.

❌ 1-naive.js — the naive approach

```
// ❌ THE NAIVE VERSION — similar object har baar scratch se, expensive setup
// duplicate ho raha hai.
// Chalao: node 1-naive.js

function createGoblin() {
  // pretend yeh expensive hai (asset load, stat calculation, etc.)
  return { type: 'goblin', hp: 30, speed: 5, loot: ['dagger'] };
}
```

```

const goblin1 = createGoblin();
const goblin2 = createGoblin();
goblin2.hp = 45; // is goblin ko thoda strong banana hai

console.log(goblin1);
console.log(goblin2);

// PROBLEM: har goblin poori setup logic dobara chalata hai, sirf ek field
// tweak karne ke liye bhi. Agar setup expensive ho (e.g. deep nested config,
// file load) to yeh wasteful hai.

```

✓ 2-prototype.js — the pattern

```

// ✓ PROTOTYPE PATTERN — ek baar "template" object banao, naye instances
// usse CLONE karke banao (expensive setup dobara nahi chalta).
// Chalao: node 2-prototype.js

// Class use karte hain taaki clone() method PROTOTYPE chain pe rahe, instance
// ki apni "own" data properties mein nahi — structuredClone(this) sirf data
// clone karta hai, functions clone nahi kar sakta (agar clone() khud ek own
// property hoti, jaisa plain object literal mein hota, to yeh crash karta).
class Goblin {
  constructor() {
    this.type = 'goblin'; this.hp = 30; this.speed = 5; this.loot = ['dagger'];
  }
  clone() {
    return structuredClone(this); // deep copy, prototype ke reference se independent
  }
}

const goblinPrototype = new Goblin(); // ek baar "expensive" setup

const goblin1 = goblinPrototype.clone();
const goblin2 = goblinPrototype.clone();
goblin2.hp = 45; // sirf clone mein change, prototype untouched

console.log(goblin1);
console.log(goblin2);
console.log('prototype still:', goblinPrototype.hp);

// Naya enemy type (orc) add karna = ek nayi class jo yehi clone() pattern
// follow kare, goblinPrototype ko CHHUE BINA.
class Orc {
  constructor() {
    this.type = 'orc'; this.hp = 60; this.speed = 3; this.loot = ['axe'];
  }
  clone() { return structuredClone(this); }
}

const orcPrototype = new Orc();
console.log(orcPrototype.clone());

```

🔗 exercise.js — your turn

```
// 🐛 EXERCISE – invoice template ko Prototype pattern se clone karo.
// Chalao: node exercise.js

// ----- naive version (isse refactor karna hai) -----
function createInvoice() {
  return {
    header: { company: 'Acme Inc', address: '123 Main St' },
    items: [],
    tax: 0.18,
  };
}

const invoiceA = createInvoice();
invoiceA.items.push({ name: 'Widget', price: 100 });
console.log(invoiceA);

// =====
// TODO 1: `InvoicePrototype` CLASS banao (constructor mein header + empty
//       items + tax set kare) with ek `clone()` method jo
//       `structuredClone(this)` return kare.
//       (Class use karo, plain object literal nahi – warna clone() khud
//       ek "own property" ban jaayegi aur structuredClone use clone nahi
//       kar payega kyunki functions clone-able nahi hote. Class method
//       prototype chain pe rehta hai, isliye safe hai – 2-prototype.js
//       mein yehi tarika dikhaya gaya hai.)
//
// TODO 2: `const invoicePrototype = new InvoicePrototype()` banao, phir do
//       invoices clone karo (`invoiceA`, `invoiceB`), dono mein alag items
//       push karo, dikhao ki ek doosre ko affect nahi karte (structuredClone
//       deep copy hai, shallow spread {...this} nahi).
//
// TODO 3 (bonus): socho – agar tumne `clone()` mein `{...this}` (shallow
//       copy) use kiya hota, to `items` array kya problem create karta?
//       (Hint: dono clones same array reference share karte.)
//
// Neeche apna solution likho, phir `node exercise.js` chala kar verify karo.
// =====
```

6. Adapter

Problem: tumhare app ko ek interface chahiye, lekin jo library/API mil rahi hai uska shape alag hai (different param order, different field names). Tum usko change nahi kar sakte (third-party/legacy).

Idea: ek thin wrapper (adapter) banao jo purani/mismatched interface ko apne app ke expected shape mein convert kare. App sirf apne expected interface se baat karta hai, adapter ke peeche kya hai — usse fark nahi padta.

Files — is order mein chalao

```
node 1-naive.js      # ❌ har jagah mismatch-handling duplicate
node 2-adapter.js    # ✅ ek wrapper, clean interface
node exercise.js     # 🎯 tumhara turn
```

Principles yahan dikh rahe hain

1. **Isolate the mismatch** — conversion logic ek hi jagah rehta hai.
2. **App depends on ITS interface, not theirs** — third-party SDK badle to sirf adapter badalta hai.

When to skip

Agar tum khud us API/library ke owner ho (usko directly badal sakte ho), adapter mat banao — seedha shape fix karo. Adapter tabhi zaroori hai jab tum kisi cheez ko control nahi kar sakte.

❌ 1-naive.js — the naive approach

```
// ❌ THE NAIVE VERSION — third-party lib ka "alag shape" wala response
// direct use ho raha hai, jahan bhi use hota hai wahi conversion duplicate.
// Chalao: node 1-naive.js

// Pretend yeh ek third-party payment SDK hai jiska shape hum control nahi
// kar sakte:
const legacyPaymentSdk = {
  makeCharge: (cents) => ({ status_code: 200, charged_amount_cents: cents }),
};
```

```
function checkout(amountRupees) {
  const result = legacyPaymentSdk.makeCharge(amountRupees * 100);
  // har jagah jahan yeh SDK use hoga, yeh conversion + shape-reading duplicate hogi
  if (result.status_code === 200) {
    console.log(`✅ Charged ₹${result.charged_amount_cents / 100}`);
  }
}

checkout(499);

// PROBLEM: baaki poora app "amount in rupees, { success, amount }" shape
// expect karta hai, lekin yeh SDK cents + status_code deta hai. Yeh
// mismatch-handling code har call-site pe copy-paste hoga.
```

✅ 2-adapter.js — the pattern

```
// ✅ ADAPTER PATTERN — third-party ke ajeeb shape ko apne app ke expected
// interface mein convert karne wala ek wrapper banao.
// Chalao: node 2-adapter.js

const legacyPaymentSdk = {
  makeCharge: (cents) => ({ status_code: 200, charged_amount_cents: cents }),
};

// Adapter — apne app ke liye "clean" interface: pay(amountRupees) -> {success, amount}
class PaymentAdapter {
  pay(amountRupees) {
    const result = legacyPaymentSdk.makeCharge(amountRupees * 100);
    return {
      success: result.status_code === 200,
      amount: result.charged_amount_cents / 100,
    };
  }
}

function checkout(amountRupees, paymentGateway) {
  const result = paymentGateway.pay(amountRupees); // apna clean interface, SDK ka pata nahi
  if (result.success) console.log(`✅ Charged ₹${result.amount}`);
}

checkout(499, new PaymentAdapter());

// Kal ko SDK badal jaaye (Stripe, Razorpay) — sirf Adapter badlega,
// checkout() ka baaki poora app CHHUE BINA rahega.
```

🔗 exercise.js — your turn

```
// 🔗 EXERCISE — purani `oldLogger` (positional args) ko naye app ke expected
// `Log({level, message})` interface mein Adapter se convert karo.
// Chalao: node exercise.js

// ----- third-party jise hum control nahi kar sakte -----
```

```

const oldLogger = {
  write: (level, msg) => console.log(`[${level.toUpperCase()}] ${msg}`),
};

// ----- naive usage (isse refactor karna hai) -----
function reportError(msg) {
  oldLogger.write('error', msg); // direct coupling – har jagah 'level, msg' order yaad rakhna
}
reportError('Payment failed');

// =====
// TODO 1: `LoggerAdapter` class banao with ek method `log({ level, message })`
//          jo internally `oldLogger.write(level, message)` call kare.
//
// TODO 2: `reportError(msg, logger)` ko refactor karo taaki woh
//          `logger.log({ level: 'error', message: msg })` call kare –
//          `oldLogger` ka naam ab reportError() ke andar kahin na aaye.
//
// TODO 3 (bonus): ek doosra adapter socho jo console.log ko hi seedha
//                  `log({level, message})` interface de (no real "old" system) –
//                  dikhao ki reportError() dono adapters ke saath kaam karta hai.
//
// Neeche apna solution likho, phir `node exercise.js` chala kar verify karo.
// =====

```

7. Bridge

Problem: ek cheez do independent dimensions mein badalti hai (e.g. urgency × channel, shape × renderer). Har combination ke liye alag class banao to combinations ke saath class count **exponentially** badhta hai.

Idea: do dimensions ko DO alag hierarchies mein tod do — ek "abstraction" (kya) aur ek "implementation" (kaise) — aur abstraction implementation ko *hold* (reference) kare, inherit nahi kare. Ab dono independently badhte hain.

Files — is order mein chalao

```
node 1-naive.js    # ❌ (urgency × channel) har combo ke liye class
node 2-bridge.js   # ✅ do hierarchies, ek doosre ko hold karti hain
node exercise.js   # 🎯 tumhara turn
```

Principles yahan dikh rahe hain

1. **Composition over inheritance** — dusri dimension ko *inherit* nahi, *hold* karo.

1. **Independent extensibility** — naya channel ya nayi urgency level add karna doosri dimension ko touch nahi karta.

When to skip

Agar sirf EK dimension hai jo badalti hai (channel ya urgency, dono nahi), Strategy pattern (Lesson 1) kaafi hai — Bridge tabhi chahiye jab do dimensions ho jo independently combine hoti hon.

❌ 1-naive.js — the naive approach

```
// ❌ THE NAIVE VERSION — har (notification-type × channel) combination ke
// liye alag class — class explosion.
// Chalao: node 1-naive.js

class UrgentEmailAlert {
  send(msg) { console.log(`📧🔥 URGENT EMAIL: ${msg}`); }
}
```

```

class NormalEmailAlert {
  send(msg) { console.log(`✉ email: ${msg}`); }
}
class UrgentSmsAlert {
  send(msg) { console.log(`📱🔥 URGENT SMS: ${msg}`); }
}
class NormalSmsAlert {
  send(msg) { console.log(`📱 sms: ${msg}`); }
}

new UrgentEmailAlert().send('Server down');
new NormalSmsAlert().send('Deploy finished');

// PROBLEM: 2 urgency levels × 2 channels = 4 classes. Teesra channel (push)
// add karo -> 6 classes. Teesri urgency level add karo -> 9 classes. Yeh
// (urgency × channel) combinations ke saath EXPONENTIALLY badhta hai.

```

✅ 2-bridge.js — the pattern

```

// ✅ BRIDGE PATTERN – do dimensions (urgency × channel) ko DO alag
// hierarchies mein tod do, ek doosre ko "hold" karke connect karo.
// Chalao: node 2-bridge.js

// Implementation hierarchy – "kaise bheja jaaye" (channel)
const emailChannel = { deliver: (text) => console.log(`✉ ${text}`) };
const smsChannel = { deliver: (text) => console.log(`📱 ${text}`) };

// Abstraction hierarchy – "kya bheja jaaye" (urgency), channel ko bas HOLD karta hai
class Alert {
  constructor(channel) { this.channel = channel; }
  send(msg) { this.channel.deliver(msg); }
}
class UrgentAlert extends Alert {
  send(msg) { this.channel.deliver(`🔥 URGENT: ${msg}`); }
}

// Ab dono dimensions INDEPENDENTLY combine hote hain – class explosion nahi
new Alert(emailChannel).send('Deploy finished');
new UrgentAlert(smsChannel).send('Server down');
new UrgentAlert(emailChannel).send('Payment failed');

// Naya channel (push) add karna = 1 naya object. Nayi urgency level add
// karna = 1 nayi class. Dono independently badhte hain, combinations nahi.

```

🎯 exercise.js — your turn

```

// 🎯 EXERCISE – Shape (circle/square) × Renderer (vector/raster) ko Bridge
// se refactor karo taaki class explosion na ho.
// Chalao: node exercise.js

// ----- naive version (isse refactor karna hai) -----
class VectorCircle { draw() { console.log('vector circle ○'); } }

```

```
class RasterCircle { draw() { console.log('raster circle (pixels) ●'); } }
class VectorSquare { draw() { console.log('vector square □'); } }
class RasterSquare { draw() { console.log('raster square (pixels) ■'); } }

new VectorCircle().draw();
new RasterSquare().draw();

// =====
// TODO 1: do "renderer" objects banao - `vectorRenderer`, `rasterRenderer` -
//         dono mein ek `renderShape(shapeName)` method ho jo shape ka naam
//         leke apne style mein console.log kare.
//
// TODO 2: `Shape` base class banao jo constructor mein renderer le, aur
//         `Circle extends Shape`, `Square extends Shape` banao jo apna naam
//         renderer ko pass karein (draw() method se).
//
// TODO 3 (bonus): naya renderer 'asciiRenderer' add karo - dikhao ki Circle
//         aur Square dono classes ko CHHUE BINA naya renderer kaam karta hai.
//
// Neeche apna solution likho, phir `node exercise.js` chala kar verify karo.
// =====
```

8. Composite

Problem: ek tree structure hai (files/folders, menu/combo, org chart) — kuch nodes "leaf" hain (single item), kuch "branch" hain (group jisme aur nodes hain). Har operation mein "leaf ya branch?" check karna padta hai.

Idea: leaf aur branch dono ko EK hi interface do (e.g. `getSize()`, `getPrice()`). Branch apna operation apne children pe recursively call karke implement karta hai. Caller ko fark hi nahi karna padta — woh bas `node.getSize()` call karta hai.

Files — is order mein chalao

```
node 1-naive.js      # ❌ har operation mein type-check duplicate
node 2-composite.js  # ✅ File aur Folder same interface follow karte hain
node exercise.js     # 🌀 tumhara turn
```

Principles yahan dikh rahe hain

1. **Uniform treatment** — single object aur group ko same tarike se treat karo.

1. **Recursive composition** — branch apna kaam apne children ko delegate karke karta hai.

When to skip

Agar structure kabhi nested nahi hoga (sirf flat list), plain array +

`.reduce()` kaafi hai — Composite ki recursion tabhi chahiye jab genuinely tree-shaped data ho.

❌ 1-naive.js — the naive approach

```
// ❌ THE NAIVE VERSION — file aur folder ko har jagah alag treat karna padta
// hai, har naye operation ke liye type-check duplicate hota hai.
// Chalao: node 1-naive.js

function getSize(node) {
  if (node.type === 'file') return node.size;
```

```

    if (node.type === 'folder') {
      let total = 0;
      for (const child of node.children) total += getSize(child); // recursion, par type-check
      saath
      return total;
    }
    throw new Error('unknown node type');
  }
}

const tree = {
  type: 'folder', name: 'src',
  children: [
    { type: 'file', name: 'index.js', size: 10 },
    { type: 'folder', name: 'utils', children: [{ type: 'file', name: 'math.js', size: 5 }] },
  ],
};

console.log('total size:', getSize(tree));

// PROBLEM: har naya operation (print, count files, search) getSize() jaisa
// har jagah "file vs folder" if/else dobara likhega. Caller ko bhi hamesha
// yaad rakhna padta hai ki node file hai ya folder.

```

✅ 2-composite.js — the pattern

```

// ✅ COMPOSITE PATTERN – File aur Folder dono EK hi interface follow karte
// hain (getSize()), caller ko fark nahi karna padta "leaf" hai ya "branch".
// Chalao: node 2-composite.js

class File {
  constructor(name, size) { this.name = name; this.size = size; }
  getSize() { return this.size; }
  print(indent = '') { console.log(`${indent} 📄 ${this.name} (${this.size})`); }
}

class Folder {
  constructor(name) { this.name = name; this.children = []; }
  add(node) { this.children.push(node); return this; }
  getSize() { return this.children.reduce((sum, c) => sum + c.getSize(), 0); } // same method
  name!
  print(indent = '') {
    console.log(`${indent} 📁 ${this.name}/`);
    this.children.forEach((c) => c.print(indent + '  '));
  }
}

const utils = new Folder('utils').add(new File('math.js', 5));
const tree = new Folder('src')
  .add(new File('index.js', 10))
  .add(utils);

tree.print();
console.log('total size:', tree.getSize());

```

```
// Naya operation (print) add karna = File aur Folder dono mein ek method,  
// caller code (`tree.print()`) file/folder ka fark kiye bina kaam karta hai.
```

exercise.js — your turn

```
// 🐛 EXERCISE – restaurant menu (single item vs combo/submenu) ko Composite  
// se refactor karo taaki total price recursively, uniformly nikle.  
// Chalao: node exercise.js  
  
// ----- naive version (isse refactor karna hai) -----  
function getPrice(entry) {  
  if (entry.kind === 'item') return entry.price;  
  if (entry.kind === 'combo') return entry.items.reduce((sum, e) => sum + getPrice(e), 0);  
  throw new Error('unknown kind');  
}  
  
const menu = {  
  kind: 'combo', name: 'Family Meal',  
  items: [  
    { kind: 'item', name: 'Burger', price: 150 },  
    { kind: 'combo', name: 'Sides', items: [{ kind: 'item', name: 'Fries', price: 50 }] },  
  ],  
};  
console.log('total price:', getPrice(menu));  
  
// =====  
// TODO 1: `MenuItem` class banao (name, price) with `getPrice()` method  
//         jo price return kare.  
//  
// TODO 2: `ComboMeal` class banao (name, empty children array) with  
//         `add(entry)` aur `getPrice()` (children ka sum, recursively –  
//         chahe entry MenuItem ho ya nested ComboMeal, same method call hota  
//         hai) methods.  
//  
// TODO 3 (bonus): dono classes mein `print(indent)` method add karo jo tree  
//                 ko nicely console.log kare (2-composite.js jaisa).  
//  
// Neeche apna solution likho, phir `node exercise.js` chala kar verify karo.  
// =====
```

9. Decorator

Problem: ek base object mein runtime pe extra behavior/features jodne hain (coffee add-ons, text formatting, middleware), lekin har combination ke liye subclass banana class-explosion ban jaata hai.

Idea: har add-on ko ek wrapper object banao jo asli object jaisa hi interface follow kare, aur wrapped object ko call karke apna kaam upar se jode. Decorators ko **kisi bhi order mein, kitne bhi stack** kiya ja sakta hai.

Files — is order mein chalao

```
node 1-naive.js      # ❌ har combination ke liye subclass
node 2-decorator.js  # ✅ stackable wrapper objects
node exercise.js     # 🎯 tumhara turn
```

Principles yahan dikh rahe hain

1. **Same interface as wrapped object** — decorator aur base dono

`cost()` / `describe()` follow karte hain, caller ko fark nahi padta.

1. **Composable at runtime** — combinations **code** mein nahi, **chaining** se banti hain.

When to skip

Agar sirf 1-2 fixed variants hain jo kabhi combine nahi hongi, simple subclass ya if/else kaafi hai — Decorator tabhi kamaata hai jab **independent add-ons ko mix-and-match** karna ho.

❌ 1-naive.js — the naive approach

```
// ❌ THE NAIVE VERSION — har add-on combination ke liye alag subclass —
// class explosion (Bridge lesson jaisa hi problem, dusre context mein).
// Chalao: node 1-naive.js

class Coffee { cost() { return 100; } describe() { return 'Coffee'; } }
class CoffeeWithMilk extends Coffee {
  cost() { return super.cost() + 20; }
```

```

    describe() { return super.describe() + ' + Milk'; }
  }
  class CoffeeWithMilkAndSugar extends CoffeeWithMilk {
    cost() { return super.cost() + 10; }
    describe() { return super.describe() + ' + Sugar'; }
  }

  const order = new CoffeeWithMilkAndSugar();
  console.log(order.describe(), '=', order.cost());

  // PROBLEM: Milk, Sugar, Whip, ExtraShot – 4 add-ons ke 2^4 = 16 possible
  // combinations, har combo ke liye subclass nahi bana sakte.

```

✓ 2-decorator.js — the pattern

```

// ✓ DECORATOR PATTERN – add-ons ko wrapper objects banao jo SAME interface
// (cost/describe) follow karte hain aur asli object ko "wrap" karte hain.
// Chalao: node 2-decorator.js

class Coffee {
  cost() { return 100; }
  describe() { return 'Coffee'; }
}

// Har decorator wahi interface (cost/describe) follow karta hai, aur jo
// wrap kiya hai uske result mein apna add-on jodta hai
class MilkDecorator {
  constructor(coffee) { this.coffee = coffee; }
  cost() { return this.coffee.cost() + 20; }
  describe() { return this.coffee.describe() + ' + Milk'; }
}
class SugarDecorator {
  constructor(coffee) { this.coffee = coffee; }
  cost() { return this.coffee.cost() + 10; }
  describe() { return this.coffee.describe() + ' + Sugar'; }
}

// Ab koi bhi combination RUNTIME pe chain ki jaa sakti hai, subclass nahi chahiye
let order = new Coffee();
order = new MilkDecorator(order);
order = new SugarDecorator(order);

console.log(order.describe(), '=', order.cost());

// Naya add-on (whip) = ek naya decorator class, existing decorators ko
// CHHUE BINA, aur kisi bhi order mein stack ho sakta hai.

```

🔗 exercise.js — your turn

```

// 🔗 EXERCISE – text formatters (bold/italic) ko Decorator pattern se
// refactor karo taaki koi bhi combination stack ho sake.
// Chalao: node exercise.js

```

```
// ----- naive version (isse refactor karna hai) -----
function boldItalic(text) { return `**_${text}_**`; }
function boldOnly(text) { return `**${text}**`; }
// Naya combo (underline+bold) chahiye? Naya function likhna padega...

console.log(boldItalic('Hello'));

// =====
// TODO 1: `PlainText` class banao with constructor(text) aur
//         `render()` method jo bas text return kare.
//
// TODO 2: `BoldDecorator` aur `ItalicDecorator` classes banao, dono
//         constructor mein "wrapped" text-object lein aur `render()` mein
//         `wrapped.render()` ke around apna marker (** ya _) laga dein.
//
// TODO 3 (bonus): `UnderlineDecorator` add karo aur teeno ko kisi bhi order
//         mein stack karke dikhao – dikhao ki order badalne se output ka
//         "nesting order" badalta hai.
//
// Neeche apna solution likho, phir `node exercise.js` chala kar verify karo.
// =====
```

10. Facade

Problem: ek kaam poora karne ke liye kai subsystems ko sahi order mein call karna padta hai (decode → normalize → encode; reserve → charge → ship).

Har caller ko yeh poora orchestration aur order yaad rakhna padta hai.

Idea: subsystem ke aage ek simple object/function rakho jo saare steps internally sahi order mein chalayen. Caller sirf ek method call karta hai, subsystem ke internals se bekhabar.

Files — is order mein chalao

```
node 1-naive.js      # ❌ caller khud subsystem orchestrate karta hai
node 2-facade.js     # ✅ ek simple entry point, internals chhupe hue
node exercise.js     # 🎯 tumhara turn
```

Principles yahan dikh rahe hain

1. **Hide complexity, don't remove it** — subsystem waisa hi rehta hai, bas uske use karne ka tarika simple ho jaata hai.

1. **Single entry point** — orchestration logic ek hi jagah, duplicate nahi hoti har caller mein.

When to skip

Agar subsystem already simple hai (1-2 calls), facade ek extra layer hi hai jo kuch nahi kamaata — sirf tab banao jab **orchestration genuinely complex** ho aur multiple callers ho.

❌ 1-naive.js — the naive approach

```
// ❌ THE NAIVE VERSION — caller ko poore subsystem (codec, audio, subtitle)
// ke saare steps aur unka sahi order khud yaad rakhna padta hai.
// Chalao: node 1-naive.js

const codec = { decode: (file) => `decoded(${file})`, encode: (data, fmt) => `${data}.${fmt}` };
const audioMixer = { normalize: (data) => `normalized(${data})` };
const subtitleEngine = { burnIn: (data, subs) => (subs ? `${data}+subs` : data) };
```

```
function convertVideo(file, format, subs) {
  const decoded = codec.decode(file);
  const normalized = audioMixer.normalize(decoded);
  const withSubs = subtitleEngine.burnIn(normalized, subs);
  return codec.encode(withSubs, format);
}

console.log(convertVideo('movie.mkv', 'mp4', true));

// PROBLEM: caller (ya iske UI/CLI) ko subsystem ke saare internal steps
// (decode -> normalize -> subs -> encode) aur unka sahi order pata hona
// chahiye. Har naya caller yeh poora orchestration dobara likhega.
```

✅ 2-facade.js — the pattern

```
// ✅ FACADE PATTERN – poore complex subsystem ke aage ek SIMPLE, single
// entry-point object rakho. Caller ko internal steps ka pata hi nahi hona
// chahiye.
// Chalao: node 2-facade.js

const codec = { decode: (file) => `decoded(${file})`, encode: (data, fmt) => `${data}.${fmt}` };
const audioMixer = { normalize: (data) => `normalized(${data})` };
const subtitleEngine = { burnIn: (data, subs) => (subs ? `${data}+subs` : data) };

// Facade – subsystem ke saare steps ko ek simple method ke peeche chhupata hai
class VideoConverterFacade {
  convert(file, format, subs = false) {
    const decoded = codec.decode(file);
    const normalized = audioMixer.normalize(decoded);
    const withSubs = subtitleEngine.burnIn(normalized, subs);
    return codec.encode(withSubs, format);
  }
}

const converter = new VideoConverterFacade();
console.log(converter.convert('movie.mkv', 'mp4', true));
console.log(converter.convert('clip.avi', 'webm'));

// Caller ko codec/audioMixer/subtitleEngine ke naam tak pata nahi – sirf
// EK simple method call karta hai. Subsystem internally reorganize ho sakta
// hai, facade ka interface same rehta hai.
```

🔗 exercise.js — your turn

```
// 🔗 EXERCISE – order placement (inventory check + payment + shipping) ko
// Facade se ek simple `placeOrder()` ke peeche chhupao.
// Chalao: node exercise.js

// ----- subsystems (already given, inhe badalna nahi hai) -----
const inventory = { reserve: (item) => console.log(`📦 reserved: ${item}`) };
const payment = { charge: (amount) => console.log(`💵 charged ₹${amount}`) };
```

```
const shipping = { schedule: (item) => console.log(`🚚 shipping scheduled: ${item}`) };

// ----- naive usage (isse refactor karna hai) -----
function buyNow(item, amount) {
  inventory.reserve(item);
  payment.charge(amount);
  shipping.schedule(item);
}
buyNow('Headphones', 1999);

// =====
// TODO 1: `OrderFacade` class banao with ek method `placeOrder(item, amount)`
//         jo internally inventory.reserve -> payment.charge -> shipping.schedule
//         (yahi order mein) call kare.
//
// TODO 2: `buyNow` function ko hata kar `new OrderFacade().placeOrder(...)`
//         use karo.
//
// TODO 3 (bonus): agar `payment.charge` fail ho jaaye (throw kare) to socho -
//                 kya inventory.reserve() undo hona chahiye? Ek naya method
//                 `inventory.release(item)` maan lo aur facade mein try/catch se
//                 rollback add karo.
//
// Neeche apna solution likho, phir `node exercise.js` chala kar verify karo.
// =====
```

11. Flyweight

Problem: bohot saare (hazaron/lakhon) similar objects banane hain, aur har object apna heavy/shared data (texture, font, config) alag se duplicate karta hai — memory waste hota hai.

Idea: jo data **shared** ho sakta hai (same type ke sab objects ke liye same) usse alag nikaal ke ek jagah reuse karo. Har instance mein sirf **unique** data (position, jaisi cheezein) rakho.

Files — is order mein chalao

```
node 1-naive.js      # ❌ har object apna shared data duplicate karta hai
node 2-flyweight.js  # ✅ shared data ek factory se reuse hota hai
node exercise.js     # 🔄 tumhara turn
```

Principles yahan dikh rahe hain

1. **Intrinsic vs extrinsic state** — intrinsic (shared, type-level) alag, extrinsic (unique, instance-level) alag.

1. **Factory ensures reuse** — naya object banane se pehle check karo ki shared version already exist karta hai kya.

When to skip

Agar objects ki count chhoti hai (sainkdo, hazaron nahi) ya shared data heavy nahi hai, yeh optimization ki zaroorat nahi — premature optimization se bachna behtar hai.

❌ 1-naive.js — the naive approach

```
// ❌ THE NAIVE VERSION — har tree object apna poora heavy "texture" data
// duplicate karta hai, hazaron trees banate hi memory phat jaayegi.
// Chalao: node 1-naive.js

class Tree {
  constructor(x, y, type) {
    this.x = x; this.y = y;
```

```

    this.type = type;
    // pretend yeh heavy shared data hai (mesh, texture...) - har tree isko duplicate karta hai
    this.texture = `${type}-texture-data-(pretend this is large)`;
    this.mesh = `${type}-mesh-data-(pretend this is large)`;
  }
  render() { console.log(` 🌳 ${this.type} at (${this.x},${this.y}) using ${this.texture}`); }
}

const forest = [];
for (let i = 0; i < 5; i++) forest.push(new Tree(i, i * 2, 'oak'));
forest.forEach((t) => t.render());
console.log('trees created:', forest.length, '- har ek apna texture+mesh copy leke');

// PROBLEM: 'oak' texture/mesh SAME hai sab trees ke liye, phir bhi har Tree
// apna alag copy rakhta hai. 1 Lakh trees banao to woh data 1 Lakh baar
// duplicate hoga - memory waste.

```

✅ 2-flyweight.js — the pattern

```

// ✅ FLYWEIGHT PATTERN - shared/heavy data (texture, mesh) ko ALAG rakho aur
// same-type objects ke beech REUSE karo. Sirf unique data (x, y) har
// instance mein rehta hai.
// Chalao: node 2-flyweight.js

// Flyweight factory - same type ke liye same shared object reuse karta hai
class TreeTypeFactory {
  static #types = new Map();
  static getType(type) {
    if (!TreeTypeFactory.#types.has(type)) {
      console.log(` (creating shared texture+mesh for '${type}' - sirf EK baar)`);
      TreeTypeFactory.#types.set(type, {
        texture: `${type}-texture-data`,
        mesh: `${type}-mesh-data`,
      });
    }
    return TreeTypeFactory.#types.get(type); // same object wapas milta hai
  }
  static count() { return TreeTypeFactory.#types.size; }
}

// Tree ab sirf UNIQUE data (x, y) rakhta hai, shared data ko reference karta hai
class Tree {
  constructor(x, y, type) {
    this.x = x; this.y = y;
    this.sharedData = TreeTypeFactory.getType(type); // reused, duplicate nahi
  }
  render() { console.log(` 🌳 tree at (${this.x},${this.y}) using ${this.sharedData.texture}`); }
}

const forest = [];
for (let i = 0; i < 5; i++) forest.push(new Tree(i, i * 2, 'oak'));
forest.forEach((t) => t.render());

```

```
console.log('unique shared objects created:', TreeTypeFactory.count(), '(sirf 1, chahe 5 trees hon)');
```

exercise.js — your turn

```
// 🐛 EXERCISE — document mein har character ka "font style" (heavy, shared)
// ko Flyweight se reuse karo, sirf position+char unique rakho.
// Chalao: node exercise.js

// ----- naive version (isse refactor karna hai) -----
class Character {
  constructor(char, x, y, font) {
    this.char = char; this.x = x; this.y = y;
    this.fontData = { family: font, size: 12, weight: 'normal' }; // har character apna copy
  }
}

const text = 'Hi';
const chars = [...text].map((c, i) => new Character(c, i, 0, 'Arial'));
console.log(chars.length, 'characters, har ek apna fontData copy leke');

// =====
// TODO 1: `FontFactory` class banao (static Map + `getFont(family)` method)
//         jo same family ke liye same shared `{family, size, weight}` object
//         return kare (2-flyweight.js jaisa).
//
// TODO 2: `Character` class ko refactor karo taaki woh apna `fontData`
//         khud na banaye, balki `FontFactory.getFont(font)` se le.
//
// TODO 3 (bonus): dikhao ki 1000 characters banane par bhi sirf 1 shared
//                 font object banta hai (console.log se count verify karo).
//
// Neeche apna solution likho, phir `node exercise.js` chala kar verify karo.
// =====
```

12. Proxy

Problem: asli object expensive hai (load slow, network call), ya usmein access-control chahiye — lekin caller ko asli object jaisa hi interface chahiye, sirf "kab aur kaise access ho" pe control chahiye.

Idea: ek "stand-in" object banao jo asli object jaisa hi interface follow kare. Proxy decide karta hai — lazily load kare, cache kare, ya access allow/deny kare — phir kaam asli object ko delegate kare.

Files — is order mein chalao

```
node 1-naive.js    # ❌ eager load, no access control
node 2-proxy.js    # ✅ lazy load + access control, same interface
node exercise.js   # 🎯 tumhara turn
```

Principles yahan dikh rahe hain

1. **Same interface as real object** — caller ko fark nahi padta proxy hai ya asli object.
1. **Control before delegate** — proxy pehle decide karta hai (load? cache? allow?), phir kaam asli object ko sonpta hai.

When to skip

Agar object cheap hai aur access-control/caching/lazy-loading ki zaroorat nahi, seedha real object use karo — Proxy sirf tab kamaata hai jab "access ko control karna" genuinely valuable ho.

❌ 1-naive.js — the naive approach

```
// ❌ THE NAIVE VERSION — heavy image har baar directly load hoti hai, chahe
// use ho ya na ho, aur koi access-control nahi.
// Chalao: node 1-naive.js

class HighResImage {
  constructor(file) {
    this.file = file;
  }
}
```

```

    console.log(`🕒 loading heavy image: ${file} (pretend this is slow)`);
  }
  display() { console.log(`🖼️ showing ${this.file}`); }
}

console.log('gallery created (but image already loaded even before display!)');
const image = new HighResImage('photo.png'); // Load turant ho gaya
image.display();
image.display(); // dobara Load nahi hota, but pehle hi ho chuka tha bina zaroorat

// PROBLEM: image object banate hi expensive load ho jaata hai, chahe display()
// kabhi call ho ya na ho. Aur koi bhi is object ko directly access kar sakta
// hai – permission check ka koi jagah nahi.

```

✅ 2-proxy.js — the pattern

```

// ✅ PROXY PATTERN – asli object ke aage ek "stand-in" rakho jo access
// control kare, aur loading ko TAB tak taale jab tak genuinely zaroorat na
// ho (lazy loading).
// Chalao: node 2-proxy.js

class HighResImage {
  constructor(file) {
    this.file = file;
    console.log(`🕒 loading heavy image: ${file} (pretend this is slow)`);
  }
  display() { console.log(`🖼️ showing ${this.file}`); }
}

// Proxy – same interface (display()) but real object ko lazily banata hai
class ImageProxy {
  #realImage = null;
  constructor(file, userRole) { this.file = file; this.userRole = userRole; }
  display() {
    if (this.userRole !== 'admin') {
      console.log('🚫 access denied – admin only');
      return;
    }
    if (!this.#realImage) this.#realImage = new HighResImage(this.file); // pehli baar hi load
    this.#realImage.display();
  }
}

console.log('gallery created (image NOT loaded yet)');
const image = new ImageProxy('photo.png', 'admin');
image.display(); // ab load hota hai
image.display(); // cached – dobara Load nahi hota

const guestImage = new ImageProxy('secret.png', 'guest');
guestImage.display(); // access denied, load hi nahi hota

```

🔗 exercise.js — your turn

```
// 🚩 EXERCISE – repeated API calls ko Proxy se cache karo taaki same request
// dobara network na kare.
// Chalao: node exercise.js

// ----- naive version (isse refactor karna hai) -----
let callCount = 0;
function fetchWeather(city) {
  callCount++;
  console.log(`🌐 network call #${callCount} for ${city}`);
  return { city, tempC: 28 };
}
fetchWeather('Delhi');
fetchWeather('Delhi'); // same city – phir bhi naya network call

// =====
// TODO 1: `WeatherProxy` class banao with internal `#cache` (Map).
//       `get(city)` method: agar cache mein hai to wahi return karo, warna
//       `fetchWeather(city)` call karke result cache mein daalo aur return
//       karo.
//
// TODO 2: proxy use karke dikhao ki 'Delhi' do baar maangne par network call
//       sirf EK baar ho.
//
// TODO 3 (bonus): `invalidate(city)` method add karo jo cache se ek city
//       hata de – dikhao ki uske baad fresh network call hota hai.
//
// Neeche apna solution likho, phir `node exercise.js` chala kar verify karo.
// =====
```

13. Strategy

Problem: ek kaam kai tarikon se ho sakta hai, aur kaunsa tarika use karna hai woh runtime pe decide hota hai (payment method, export format, shipping cost...).

Idea: har variant ko apne function/object mein nikaalo jo ek **common contract** follow kare. Caller ko sirf woh reference pakdao jo use chahiye. Naya variant add karna = naya object likhna, purana code chhue bina.

Files — is order mein chalao

```
node 1-naive.js          # ❌ if/else wali problem
node 2-strategy-class.js # ✅ class + strategy objects
node 3-strategy-registry.js # ✅ idiomatic JS: object as registry (best for JS)
node exercise.js         # 🔄 tumhara turn
```

Teen principles yahan dikh rahe hain

1. **Encapsulate what varies** — har payment method ka logic alag.
2. **Program to an interface** — sab `pay(amount)` contract follow karte hain.
3. **Composition over inheritance** — Checkout strategy ko **hold** karta hai, inherit nahi karta.

When to skip

Sirf 2 variants hain jo kabhi badhenge nahi? Plain `if` behtar — pattern ki indirection ki cost mat uthao.

❌ 1-naive.js — the naive approach

```
// ❌ THE NAIVE VERSION — sab kuch ek function mein if/else ke saath.
// Chalao: node 1-naive.js

// Mock payment services (pretend ye real APIs call karte hain)
const processStripe = (amt) => `✅ Charged ₹${amt} via Stripe`;
const processPaypal = (amt) => `✅ Charged ₹${amt} via PayPal`;
const processCrypto = (amt) => `✅ Charged ₹${amt} via Crypto`;

function checkout(cart, method) {
  if (method === 'card') return processStripe(cart.total);
  if (method === 'paypal') return processPaypal(cart.total);
}
```

```

    if (method === 'crypto') return processCrypto(cart.total);
    throw new Error(`Unknown method: ${method}`);
  }

  const cart = { total: 4999 };
  console.log(checkout(cart, 'card'));
  console.log(checkout(cart, 'paypal'));
  console.log(checkout(cart, 'crypto'));

  // PROBLEM: naya method (Apple Pay) add karne ke liye is function ko DOBARA
  // kholna padega. Jitne payment ho sakte hain, utni jagah ye if/else copy hota
  // hai. Ek path ko akele test karna bhi mushkil. => Strategy pattern isse theek karta hai.

```

✅ 2-strategy-class.js — the pattern

```

// ✅ STRATEGY PATTERN (class form)
// Chalao: node 2-strategy-class.js

const processStripe = (amt) => `✅ Charged ₹${amt} via Stripe`;
const processPaypal = (amt) => `✅ Charged ₹${amt} via PayPal`;
const processCrypto = (amt) => `✅ Charged ₹${amt} via Crypto`;

// Har strategy ek hi contract follow karti hai: pay(amount)
const cardStrategy = { pay: (amount) => processStripe(amount) };
const paypalStrategy = { pay: (amount) => processPaypal(amount) };
const cryptoStrategy = { pay: (amount) => processCrypto(amount) };

// "Context" sirf delegate karta hai — use pata nahi kaunsi strategy hai
class Checkout {
  constructor(strategy) { this.strategy = strategy; }
  setStrategy(strategy) { this.strategy = strategy; }
  complete(amount) { return this.strategy.pay(amount); }
}

const checkout = new Checkout(cardStrategy);
console.log(checkout.complete(4999));

// Runtime pe behavior swap:
checkout.setStrategy(cryptoStrategy);
console.log(checkout.complete(4999));

// Naya method add karna = naya object, Checkout ko CHHUE BINA:
const applePayStrategy = { pay: (amount) => `✅ Charged ₹${amount} via Apple Pay` };
checkout.setStrategy(applePayStrategy);
console.log(checkout.complete(4999));

```

✅ 3-strategy-registry.js — the pattern

```

// ✅ IDIOMATIC JS: object-as-registry. JS mein aksar YEHI best hai.
// Chalao: node 3-strategy-registry.js

const processStripe = (amt) => `✅ Charged ₹${amt} via Stripe`;

```

```

const processPaypal = (amt) => `✅ Charged ₹${amt} via PayPal`;
const processCrypto = (amt) => `✅ Charged ₹${amt} via Crypto`;

// Functions first-class hain => strategy bas ek function, registry bas ek object.
const strategies = {
  card: (amount) => processStripe(amount),
  paypal: (amount) => processPaypal(amount),
  crypto: (amount) => processCrypto(amount),
};

function checkout(method, amount) {
  const pay = strategies[method];
  if (!pay) throw new Error(`Unknown method: ${method}`);
  return pay(amount);
}

console.log(checkout('card', 4999));
console.log(checkout('paypal', 4999));

// Apple Pay add karna = EK line. if/else ko chhue bina:
strategies.applePay = (amount) => `✅ Charged ₹${amount} via Apple Pay`;
console.log(checkout('applePay', 4999));

// Yahi tarika hai jisse experienced JS dev naturally if/else chain khatam karta hai.

```

exercise.js — your turn

```

// 🛠️ EXERCISE — ise Strategy pattern (registry form) mein refactor karo.
// Chalao: node exercise.js
//
// Rules:
//   'standard' -> flat ₹50
//   'express' -> ₹50 + ₹10 per kg
//   'overnight' -> ₹200 flat

// ----- naive version (isse refactor karna hai) -----
function shippingCost(order, method) {
  if (method === 'standard') return 50;
  if (method === 'express') return 50 + 10 * order.weightKg;
  if (method === 'overnight') return 200;
  throw new Error(`Unknown method: ${method}`);
}

const order = { weightKg: 3, total: 1200 };
console.log('standard:', shippingCost(order, 'standard'));
console.log('express:', shippingCost(order, 'express'));
console.log('overnight:', shippingCost(order, 'overnight'));

// =====
// TODO 1: upar wale ko `const strategies = { ... }` + chhote dispatch
//         function mein badlo. Har strategy ko (order) milega.
//
// TODO 2 (bonus): 'express-free-above-1000' add karo — agar order.total > 1000

```

```
//      to express free (₹0), warna normal express charge.
//      Sawaal: naya rule add karte waqt kya PURANI strategies ko chhuna pada?
//      (Sahi jawab: nahi. Yehi open/closed ka fayda hai.)
//
// Neeche apna solution likho, phir `node exercise.js` chala kar verify karo.
// =====

// const strategies = {
//   ...
// };
// function shipping(method, order) { ... }
// console.log(shipping('express', order));
```

14. Observer

Problem: ek object ki state badalti hai, aur **kai doosre** objects ko react karna hai — par source ko yeh nahi jaanna chahiye ki kaun sun raha hai.
(order shipped → email + SMS + analytics + inventory...)

Idea: Subject sirf ek list rakhta hai interested logon ki (Observers), aur change pe unhe **notify** kar deta hai — bina jaane woh karte kya hain.

Files — is order mein chalao

```
node 1-naive.js      # ❌ Order har service se coupled
node 2-observer.js   # ✅ subscribe / notify / unsubscribe khud banaya
node 3-eventemitter.js # ✅ Node ka built-in EventEmitter = Observer pattern
node exercise.js     # 🎯 tumhara turn
```

Ye JS mein built-in hai

- Node: `EventEmitter` — `on` = subscribe, `emit` = notify, `off` = unsubscribe
- Browser: `addEventListener` / `dispatchEvent`

2 khatre (exercise mein dhyaan do)

1. **Execution order guarantee nahi** — agar B ko A ke baad chalna zaroori hai, Observer galat tool hai.

1. **Memory leak** — `unsubscribe` bhoole to subject observer ko pakde rehta hai aur woh garbage-collect nahi hota.

❌ 1-naive.js — the naive approach

```
// ❌ THE NAIVE VERSION — Order khud sabko directly call karta hai.
// Chalao: node 1-naive.js

const sendShippedEmail = (o) => console.log(`✉ Email: order ${o.id} shipped`);
const smsNotify         = (o) => console.log(`📱 SMS: order ${o.id} shipped`);
const analyticsTrack     = (evt, o) => console.log(`📊 Analytics: ${evt} (${o.id})`);
const inventoryDecrement = (n) => console.log(`📦 Inventory down by ${n} items`);

class Order {
  constructor(id, items) { this.id = id; this.items = items; }
```

```

ship() {
  this.status = 'shipped';
  // Order ab in SAB services ke baare mein jaanta hai – tight coupling
  sendShippedEmail(this);
  smsNotify(this);
  analyticsTrack('order_shipped', this);
  inventoryDecrement(this.items);
}
}

new Order('A1', 2).ship();

// PROBLEM: naya reaction (loyalty points) add karna = Order.ship() ko dobara
// kholna. Order test karne ke liye ye sab mock karne padenge. Dependency ulti hai:
// core business object apne peripheral consumers ko jaanta hai.

```

✓ 2-observer.js — the pattern

```

// ✓ OBSERVER PATTERN (khud banaya)
// Chalao: node 2-observer.js

class Subject {
  #observers = new Set();

  // subscribe ek "unsubscribe" function return karta hai – cleanup aasan
  subscribe(observer) {
    this.#observers.add(observer);
    return () => this.unsubscribe(observer);
  }
  unsubscribe(observer) { this.#observers.delete(observer); }
  notify(data) {
    for (const observer of this.#observers) observer.update(data);
  }
}

class Order extends Subject {
  constructor(id, items) { super(); this.id = id; this.items = items; }
  ship() {
    this.status = 'shipped';
    this.notify({ event: 'shipped', order: this }); // bas. kaun sun raha, pata nahi.
  }
}

// Har observer sirf ek contract follow karta hai: update(data)
const emailObserver = { update: ({ order }) => console.log(`✉ Email: ${order.id}`) };
const inventoryObserver = { update: ({ order }) => console.log(`📦 Inventory: -${order.items}`) };

const order = new Order('A1', 2);
order.subscribe(emailObserver);
const stopInventory = order.subscribe(inventoryObserver);

order.ship(); // dono chalte hain

```

```

console.log('--- inventory unsubscribe karne ke baad ---');
stopInventory();           // ab inventory nahi sunega
order.ship();              // sirf email chalega

// Naya reaction add karna = naya observer + subscribe. Order ko CHHUE BINA.

```

✅ 3-eventemitter.js — the pattern

```

// ✅ IDIOMATIC JS: Node ka EventEmitter = Observer pattern, built-in.
// Chalao: node 3-eventemitter.js

import { EventEmitter } from 'node:events';

class Order extends EventEmitter {
  constructor(id, items) { super(); this.id = id; this.items = items; }
  ship() {
    this.status = 'shipped';
    this.emit('shipped', this); // notify()
  }
}

const order = new Order('A1', 2);

order.on('shipped', (o) => console.log(`📧 Email: ${o.id}`)); // subscribe
order.on('shipped', (o) => console.log(`📦 Inventory: -${o.items}`)); // subscribe

const analytics = (o) => console.log(`📊 Analytics: ${o.id}`);
order.on('shipped', analytics);

order.ship();

console.log('--- analytics off karne ke baad ---');
order.off('shipped', analytics); // unsubscribe
order.ship();

// on = subscribe, emit = notify, off = unsubscribe. Naam alag, pattern wahi.

```

🔧 exercise.js — your turn

```

// 🧑‍🔧 EXERCISE – ek StockPrice subject banao Observer pattern se.
// Chalao: node exercise.js
//
// Chahiye:
// - logger: har price change print kare      ("📈 Price: 105")
// - alerter: sirf tab print kare jab price 100 ke UPAR cross kare ("⚠️ ALERT: 105")
//
// Do tarike se try karo (dono seekhne layak):
// A) khud Subject class banake (dekho 2-observer.js)
// B) EventEmitter extend karke (dekho 3-eventemitter.js)

// ---- yahan implement karo ----
class StockPrice /* extends ??? */ {

```

```
constructor() {
  this.price = 0;
}
setPrice(newPrice) {
  // TODO: purani price yaad rakho (cross detect karne ke liye)
  // TODO: price update karo
  // TODO: observers ko notify karo
}
}

// ---- test (implement karne ke baad uncomment karo) ----
// const stock = new StockPrice();
// stock.subscribe(/* Logger */);
// stock.subscribe(/* alerter */);
// stock.setPrice(95); // logger: 95
// stock.setPrice(105); // logger: 105 + alerter: ⚠ ALERT (100 cross hua)
// stock.setPrice(110); // logger: 110 (koi alert nahi – pehle se upar tha)

console.log('Exercise file – apna code likho aur test uncomment karo.');
```

15. Command

Problem: ek "action" (button press, menu click, editor edit) ko turant execute karna hai, lekin baad mein **undo karna, queue mein daalna**, ya **log karke replay karna** bhi chahiye — aur caller (remote/UI) ko device ke internals nahi pata hone chahiye.

Idea: action ko khud ek object bana do — `execute()` aur `undo()` wala.

Invoker (remote/menu) sirf command ko **hold** karta hai aur call karta hai, uska implementation nahi jaanta.

Files — is order mein chalao

```
node 1-naive.js      # ❌ direct method calls, undo impossible
node 2-command.js    # ✅ Command objects + undo
node 3-command-queue.js # ✅ macro, queue, replay
node exercise.js     # 🎯 tumhara turn
```

Principles yahan dikh rahe hain

1. **Encapsulate a request as an object** — action + uska undo ek jagah.
2. **Decouple invoker from receiver** — RemoteControl ko Light ka pata nahi.
3. Ek baar action object ban jaaye to use **queue, log, ya batch** kar sakte ho.

When to skip

Agar undo/queue/log kabhi chahiye hi nahi, aur action seedha ek function call hai — seedha call karo. Command ki indirection tabhi kamaati hai jab in extra powers (undo/redo, macros, replay) ki zaroorat ho.

❌ 1-naive.js — the naive approach

```
// ❌ THE NAIVE VERSION — remote directly calls device methods, no undo.
// Chalao: node 1-naive.js

const light = {
  on: () => console.log('💡 Light: ON'),
  off: () => console.log('💡 Light: OFF'),
};
```

```
function pressButton(action) {
  if (action === 'light-on') return light.on();
  if (action === 'light-off') return light.off();
  throw new Error(`Unknown action: ${action}`);
}

pressButton('light-on');
pressButton('light-off');

// PROBLEM: remote ko device ke internals pata hone chahiye. Undo impossible -
// Last action record nahi ho raha. Naya device (fan, AC) add karo to
// pressButton() dobara edit karna padega. Command pattern isse theek karta hai.
```

✅ 2-command.js — the pattern

```
// ✅ COMMAND PATTERN - action ko object bana do (execute + undo).
// Chalao: node 2-command.js

const light = {
  on: () => console.log('💡 Light: ON'),
  off: () => console.log('💡 Light: OFF'),
};

// Har command ek hi contract follow karta hai: execute() aur undo()
class LightOnCommand {
  execute() { light.on(); }
  undo() { light.off(); }
}
class LightOffCommand {
  execute() { light.off(); }
  undo() { light.on(); }
}

// Invoker - kisi command ke internals nahi jaanta, bas execute/undo call karta hai
class RemoteControl {
  #history = [];
  press(command) {
    command.execute();
    this.#history.push(command);
  }
  undoLast() {
    const command = this.#history.pop();
    if (command) command.undo();
  }
}

const remote = new RemoteControl();
remote.press(new LightOnCommand());
remote.press(new LightOffCommand());
remote.undoLast(); // Last action (off) undo -> light wapas ON

// Naya device (fan) add karna = naye Command classes, RemoteControl ko CHHUE BINA.
```

✓ 3-command-queue.js — the pattern

```
// ✓ COMMAND PATTERN – queue, macro aur replay (Log ke through).
// Chalao: node 3-command-queue.js

const light = { on: () => console.log('💡 Light: ON'), off: () => console.log('💡 Light: OFF')
};
const fan    = { on: () => console.log('🌀 Fan: ON'),   off: () => console.log('🌀 Fan: OFF') };

const cmd = (device, action) => ({
  execute: () => device[action](),
  label: `${action} ${device === light ? 'light' : 'fan'}`,
});

// Macro command = kai commands ka ek hi command
class MacroCommand {
  constructor(commands) { this.commands = commands; }
  execute() { this.commands.forEach((c) => c.execute()); }
}

const goodMorning = new MacroCommand([
  cmd(light, 'on'),
  cmd(fan, 'on'),
]);

console.log('-- Good morning macro --');
goodMorning.execute();

// Queue – commands ko turant execute karne ke bajaye baad ke liye rakho
const queue = [];
queue.push(cmd(light, 'off'));
queue.push(cmd(fan, 'off'));

console.log('-- Draining queue (e.g. batch job / scheduled task) --');
while (queue.length) queue.shift().execute();

// Replay – har executed command ka log rakho, baad mein wahi sequence dobara chalao
const log = [cmd(light, 'on'), cmd(fan, 'on'), cmd(light, 'off')];
console.log('-- Replaying log --');
log.forEach((c) => c.execute());
```

🔗 exercise.js — your turn

```
// 🔗 EXERCISE – text editor ke type/delete actions ko Command pattern mein
// refactor karo taaki undo() kaam kare.
// Chalao: node exercise.js

// ----- naive version (isse refactor karna hai) -----
let text = '';
function type(chars) { text += chars; console.log('text:', JSON.stringify(text)); }
function deleteLast(n) { text = text.slice(0, -n); console.log('text:', JSON.stringify(text)); }
```

```
type('Hello');
type(' ', world');
deleteLast(7); // 'undo' karna ho to yaad nahi ki kitna delete hua tha

// =====
// TODO 1: TypeCommand aur DeleteCommand classes banao, dono mein execute()
//         aur undo() ho. TypeCommand.undo() = wahi characters hata de jo
//         type kiye the. DeleteCommand.undo() = jo delete hua wapas jod de
//         (execute se pehle un characters ko save kar lo).
//
// TODO 2: Editor class banao jisme history array ho, type()/deleteLast()
//         commands create/execute/push karein, aur undo() history se pop
//         karke us command ka undo() chalaye.
//
// TODO 3 (bonus): redo() add karo – undo kiye hue commands ek "redo stack"
//         mein daalo, redo() unhe wapas execute kare.
//
// Neeche apna solution likho, phir `node exercise.js` chala kar verify karo.
// =====
```

16. Chain of Responsibility

Problem: ek request ko kai possible handlers mein se koi ek handle kar sakta hai (approval levels, support ticket routing, middleware), aur "kaun handle karega" ka decision nested if/else mein tightly coupled hai.

Idea: har handler ko ek chain mein jodo. Har handler khud decide kare — "main isse handle kar sakta hoon?" Agar haan, kaam khatam. Agar nahi, request ko **agle handler** ko pass kar do.

Files — is order mein chalao

node 1-naive.js	# ❌ ek function, nested if/else levels
node 2-chain-of-responsibility.js	# ✅ handlers ki chain, har ek apna decision leta hai
node exercise.js	# 🎯 tumhara turn

Principles yahan dikh rahe hain

1. **Decouple sender from receiver** — sender ko pata nahi kaun final handle karega.

1. **Open/closed** — naya handler = chain mein ek naya link, existing handlers ko CHHUE BINA.

When to skip

Agar handlers ka order/set fixed hai aur kabhi badlega nahi, seedha if/else ya switch simpler hai — chain tabhi kamaati hai jab handlers ****dynamically configure/reorder**** hone chahiye.

❌ 1-naive.js — the naive approach

```
// ❌ THE NAIVE VERSION — approval levels ek hi function mein nested if/else,
// naya level add karna is function ko dobara edit karna hai.
// Chalao: node 1-naive.js

function approveExpense(amount) {
  if (amount <= 1000) {
    console.log(`✅ Manager approved ₹${amount}`);
```

```

    } else if (amount <= 10000) {
      console.log(`✅ Director approved ₹${amount}`);
    } else if (amount <= 100000) {
      console.log(`✅ VP approved ₹${amount}`);
    } else {
      console.log(`❌ ₹${amount} needs CEO approval – not implemented here`);
    }
  }
}

approveExpense(500);
approveExpense(5000);
approveExpense(50000);

// PROBLEM: naya approval level (CEO) ya level ke thresholds badalne ke liye
// is function ko dobara edit karna padega. Poora "kaun approve karta hai"
// logic ek hi jagah tightly coupled hai.

```

✅ 2-chain-of-responsibility.js — the pattern

```

// ✅ CHAIN OF RESPONSIBILITY – har approver apna "handle kar sakta hoon ya
// nahi" khud decide kare, nahi to REQUEST ko agle handler ko pass kar de.
// Chalao: node 2-chain-of-responsibility.js

class Approver {
  constructor(name, limit) { this.name = name; this.limit = limit; this.next = null; }
  setNext(approver) { this.next = approver; return approver; }
  handle(amount) {
    if (amount <= this.limit) {
      console.log(`✅ ${this.name} approved ₹${amount}`);
    } else if (this.next) {
      this.next.handle(amount); // apne bas ka nahi, agle ko pass kar do
    } else {
      console.log(`❌ ₹${amount} – koi bhi approve nahi kar saka`);
    }
  }
}

const manager = new Approver('Manager', 1000);
const director = new Approver('Director', 10000);
const vp = new Approver('VP', 100000);
manager.setNext(director).setNext(vp); // chain banao

manager.handle(500);
manager.handle(5000);
manager.handle(50000);
manager.handle(500000); // koi handle nahi kar sakta

// Naya Level (CEO, unlimited) add karna = ek naya Approver, chain mein jodo,
// baaki approvers ko CHHUE BINA.

```

🔗 exercise.js — your turn

```

// 🚩 EXERCISE – support ticket routing ko Chain of Responsibility se
// refactor karo (bot -> junior agent -> senior agent).
// Chalao: node exercise.js

// ----- naive version (isse refactor karna hai) -----
function routeTicket(ticket) {
  if (ticket.type === 'faq') console.log('🤖 Bot answered:', ticket.question);
  else if (ticket.priority === 'low') console.log('😓 Junior agent handling:', ticket.question);
  else console.log('👤 Senior agent handling:', ticket.question);
}

routeTicket({ type: 'faq', question: 'What are your hours?' });
routeTicket({ type: 'issue', priority: 'high', question: 'Server is down!' });

// =====
// TODO 1: `Handler` base class banao (jaisa Approver 2-chain-of-
//       responsibility.js mein tha) with setNext(), aur `handle(ticket)`
//       jo `canHandle(ticket)` check kare, warna next ko pass kare.
//
// TODO 2: `BotHandler` (canHandle: type === 'faq'), `JuniorHandler`
//       (canHandle: priority === 'low'), `SeniorHandler` (canHandle:
//       hamesha true – akhri fallback) banao.
//
// TODO 3 (bonus): chain build karo aur dono example tickets route karo,
//       verify karo ki sahi handler pick ho raha hai.
//
// Neeche apna solution likho, phir `node exercise.js` chala kar verify karo.
// =====

```

17. Iterator

Problem: alag-alag collections (array, linked list, tree) ka internal structure alag hota hai. Agar caller ko har collection traverse karne ke liye uska internal shape jaanna pade, to generic "loop over anything" code likhna mushkil ho jaata hai.

Idea: har collection apna traversal khud implement kare, lekin bahar se ek **uniform interface** (`next()` / JS ka `Symbol.iterator`) dikhaye. Caller sirf `for...of` use karta hai — collection array hai ya linked list, usse fark nahi padta.

Files — is order mein chalao

```
node 1-naive.js      # ❌ har collection ka traversal alag, caller ko internals pata hone chahiye
node 2-iterator.js   # ✅ Symbol.iterator — uniform `for...of` sabke liye
node exercise.js     # 🔄 tumhara turn
```

Principles yahan dikh rahe hain

1. **Uniform traversal interface** — caller collection ke internals se decouple ho jaata hai.

1. JS mein `Symbol.iterator` implement karna matlab tumhara custom type `for...of`, `spread ( [...x] )`, `Array.from()` sab ke saath kaam karta hai.

When to skip

JS ke built-in collections (Array, Map, Set) already iterable hain — apna Iterator sirf tab likho jab tumhara **custom data structure** (linked list, tree, graph) ho jise `for...of` se traverse karna ho.

❌ 1-naive.js — the naive approach

```
// ❌ THE NAIVE VERSION — caller ko har collection ke internal structure ka
// pata hona chahiye, taaki sahi tarike se loop kare.
// Chalao: node 1-naive.js
```

```

const arrayCollection = [1, 2, 3];
const linkedList = { head: { value: 'a', next: { value: 'b', next: { value: 'c', next: null } } } };

// Array ke liye ek tarika
for (const item of arrayCollection) console.log('array item:', item);

// Linked list ke liye BILKUL alag tarika – caller ko internal shape pata hona zaroori
let node = linkedList.head;
while (node) { console.log('list item:', node.value); node = node.next; }

// PROBLEM: caller ko har collection type ka internal structure yaad rakhna
// padta hai. Ek generic function jo "kisi bhi collection" pe kaam kare –
// likhna mushkil hai kyunki traversal logic collection ke saath tightly
// coupled hai.

```

✅ 2-iterator.js — the pattern

```

// ✅ ITERATOR PATTERN – har collection apna traversal khud implement kare,
// lekin BAHAR se ek hi uniform interface (`next()`) dikhaye. Caller ko
// internal structure jaanne ki zaroorat nahi.
// Chalao: node 2-iterator.js

class LinkedList {
  constructor() { this.head = null; }
  push(value) {
    const node = { value, next: null };
    if (!this.head) { this.head = node; return this; }
    let cur = this.head;
    while (cur.next) cur = cur.next;
    cur.next = node;
    return this;
  }
  // Iterator protocol – JS ke `for...of` se compatible!
  [Symbol.iterator]() {
    let current = this.head;
    return {
      next: () => {
        if (!current) return { done: true, value: undefined };
        const value = current.value;
        current = current.next;
        return { done: false, value };
      },
    };
  }
}

const list = new LinkedList().push('a').push('b').push('c');

// Ab caller ko linked-list ke internals (head/next) ka pata hi nahi –
// array ki tarah hi `for...of` use karta hai:
for (const item of list) console.log('list item:', item);

```

```
function printAll(collection) {
  for (const item of collection) console.log(' ->', item);
}
printAll([1, 2, 3]); // array
printAll(list);      // Linked List – SAME function, dono pe kaam karta hai!
```

exercise.js — your turn

```
// 🧠 EXERCISE – ek binary tree ko Iterator pattern se traverse-able banao,
// taaki `for...of` se depth-first traverse ho sake.
// Chalao: node exercise.js

// ----- given structure (isse badalna nahi hai) -----
const tree = {
  value: 1,
  left: { value: 2, left: { value: 4, left: null, right: null }, right: null },
  right: { value: 3, left: null, right: null },
};

// ----- naive version (isse refactor karna hai) -----
function printTree(node) {
  if (!node) return;
  console.log(node.value);
  printTree(node.left);
  printTree(node.right);
}
printTree(tree);

// =====
// TODO 1: `TreeNode` class banao (value, left, right) jismein
//         `[Symbol.iterator]()` method ho jo depth-first (pre-order:
//         self -> left -> right) traversal kare aur `{done, value}` objects
//         de (2-iterator.js jaisa pattern).
//         Hint: recursion se ek array collect karke uska iterator return
//         karna sabse simple hoga.
//
// TODO 2: upar wale plain-object `tree` ko `TreeNode` instances se dobara
//         banao, phir `for (const v of root) console.log(v)` se print karo.
//
// TODO 3 (bonus): ek "breadth-first" iterator variant socho – kya isse bhi
//         `Symbol.iterator` ke through expose kiya jaa sakta hai? (Hint:
//         alag method naam do, jaise `bfs()`, jo khud ek iterable return kare.)
//
// Neeche apna solution likho, phir `node exercise.js` chala kar verify karo.
// =====
```

18. Mediator

Problem: components (UI fields, chat users, game objects) ek doosre ko DIRECTLY reference karte hain taaki interact kar sakein. Jitne components, wiring utni hi complex (many-to-many) ho jaati hai — naya component add karna sabko touch karwata hai.

Idea: components ek doosre ko na jaanein, sab sirf ek **central mediator** se baat karein. Mediator hi decide karta hai ki kisi event pe kaun kya karega.

Files — is order mein chalao

```
node 1-naive.js      # ❌ components ek doosre ko directly reference karte hain
node 2-mediator.js   # ✅ sab sirf mediator se baat karte hain
node exercise.js     # 🎯 tumhara turn
```

Principles yahan dikh rahe hain

1. **Decouple components from each other** — koi bhi component doosre ka naam tak nahi jaanta.

1. **Centralize interaction logic** — "kaun kisse affect hota hai" ka rule ek hi jagah (mediator) rehta hai.

When to skip

Agar sirf 2 components hain jinka relationship simple aur permanent hai, direct reference likhna hi simpler hai — Mediator tabhi kamaata hai jab **kai components ka wiring complex/changing** ho.

❌ 1-naive.js — the naive approach

```
// ❌ THE NAIVE VERSION — form fields ek doosre ko DIRECTLY reference karte
// hain — many-to-many wiring, naya field add karna sabko edit karwata hai.
// Chalao: node 1-naive.js

class Checkbox {
  constructor() { this.checked = false; }
```

```

toggle(submitButton) {
  this.checked = !this.checked;
  console.log('checkbox:', this.checked);
  submitButton.setEnabled(this.checked); // checkbox ko SubmitButton ka pata hai
}
}

class SubmitButton {
  setEnabled(enabled) { console.log('submit button enabled:', enabled); }
}

const submitButton = new SubmitButton();
const checkbox = new Checkbox();
checkbox.toggle(submitButton);
checkbox.toggle(submitButton);

// PROBLEM: Checkbox ko SubmitButton ke baare mein directly pata hai. Ek
// teesra field (dropdown) add karo jo bhi submit ko affect kare – ab
// Checkbox aur Dropdown dono ko ek doosre + SubmitButton sabka pata hona
// padega. Components ka wiring  $O(n^2)$  ho jaata hai.

```

✓ 2-mediator.js — the pattern

```

// ✓ MEDIATOR PATTERN – components ek doosre ko directly nahi jaante, sab
// ek CENTRAL mediator se baat karte hain. Mediator decide karta hai "kisko
// kya karna hai".
// Chalao: node 2-mediator.js

class FormMediator {
  #components = {};
  register(name, component) { this.#components[name] = component; component.mediator = this; }
  notify(sender, event) {
    if (sender === 'checkbox' && event === 'toggled') {
      const checked = this.#components.checkbox.checked;
      this.#components.submitButton.setEnabled(checked);
    }
    // naya field/rule add karna = yahan ek naya `if`, components ko chhue bina
  }
}

class Checkbox {
  constructor() { this.checked = false; }
  toggle() {
    this.checked = !this.checked;
    console.log('checkbox:', this.checked);
    this.mediator.notify('checkbox', 'toggled'); // sirf mediator ko batata hai, SubmitButton ko
    // nahi jaanta
  }
}

class SubmitButton {
  setEnabled(enabled) { console.log('submit button enabled:', enabled); }
}

const mediator = new FormMediator();
const checkbox = new Checkbox();

```

```

const submitButton = new SubmitButton();
mediator.register('checkbox', checkbox);
mediator.register('submitButton', submitButton);

checkbox.toggle();
checkbox.toggle();

// Naya field (dropdown) add karna = mediator mein ek naya `if`/register call,
// Checkbox aur SubmitButton ko ek doosre ke baare mein KUCH pata nahi hona chahiye.

```

exercise.js — your turn

```

// 🧠 EXERCISE – chat users ko Mediator (ChatRoom) se connect karo taaki wo
// ek doosre ko directly reference na karein.
// Chalao: node exercise.js

// ----- naive version (isse refactor karna hai) -----
class User {
  constructor(name) { this.name = name; }
  sendDirect(otherUser, msg) { // har user ko doosre users ka direct reference chahiye
    console.log(`[${this.name} -> ${otherUser.name}]: ${msg}`);
  }
}

const alice = new User('Alice');
const bob = new User('Bob');
alice.sendDirect(bob, 'Hi Bob!');

// =====
// TODO 1: `ChatRoom` class banao with `register(user)` (users array mein
//         add + user.room = this set kare) aur `broadcast(sender, msg)`
//         (sender ke alawa sabko print kare: `[sender -> everyone]: msg`).
//
// TODO 2: `User` class ko refactor karo – `send(msg)` method ho jo
//         `this.room.broadcast(this, msg)` call kare, koi doosre User ka
//         direct reference na ho.
//
// TODO 3 (bonus): teesra user 'Carol' add karo, dikhao ki Alice ka message
//                 Bob aur Carol dono ko milta hai – Alice ko unka direct reference
//                 kabhi nahi pata.
//
// Neeche apna solution likho, phir `node exercise.js` chala kar verify karo.
// =====

```

19. Memento

Problem: ek object ka state kabhi-kabhi save aur baad mein restore karna hai (undo, save-game, checkpoints) — lekin caller ko object ke internal fields khud copy/restore karne padte hain, jo encapsulation todta hai aur internals badalne par toot jaata hai.

Idea: object khud apna snapshot banaye (`save()`) aur khud restore kare (`restore()`). Caller sirf snapshot ko *opaque* tareeke se hold karta hai — uske andar kya hai, usse matlab nahi.

Files — is order mein chalao

```
node 1-naive.js      # ❌ caller directly internal fields copy/restore karta hai
node 2-memento.js    # ✅ object khud save()/restore() implement karta hai
node exercise.js     # 🎯 tumhara turn
```

Principles yahan dikh rahe hain

1. **Encapsulation preserved** — internals sirf object khud dekhta/badalta hai, caller nahi.
1. **Snapshot is opaque to the caller** — History/SaveSlot bas snapshot ko store karte hain, uske andar jhaankte nahi.

Related: Command se fark

Command "actions" ko undo karta hai (execute/undo pairs). Memento "state" ko undo karta hai (poora snapshot save/restore). Command tab better hai jab undo logic simple ho; Memento tab better hai jab state complex ho aur "reverse action" likhna mushkil ho.

When to skip

Agar state bohot chhota/simple hai (1-2 primitive fields), seedha ek variable mein copy rakhna kaafi hai — Memento tabhi kamaata hai jab state

complex ho aur encapsulation todna avoid karna ho.

❌ 1-naive.js — the naive approach

```
// ❌ THE NAIVE VERSION – state ko bahar se directly copy/restore kiya ja
// raha hai – editor ke internal fields sabko expose karne padte hain.
// Chalao: node 1-naive.js

class Editor {
  constructor() { this.text = ''; }
  type(chars) { this.text += chars; }
}

const editor = new Editor();
editor.type('Hello');

// Undo ke liye caller khud state copy kar raha hai – editor ke internals
// (this.text) directly access ho rahe hain
const backup = editor.text; // agar Editor mein aur fields add ho (cursor, formatting)
// to yeh backup INCOMPLETE ho jaayega

editor.type(', world');
console.log('before undo:', editor.text);
editor.text = backup; // caller directly internal field ko overwrite kar raha hai
console.log('after undo:', editor.text);

// PROBLEM: caller ko Editor ke internal fields ka pata hona chahiye aur
// undo logic khud implement karna padta hai. Editor ke internals badle
// (naya field add ho) to yeh backup/restore code bhi TOOT sakta hai.
```

✅ 2-memento.js — the pattern

```
// ✅ MEMENTO PATTERN – object khud apna "snapshot" banaye aur khud restore
// kare. Caller ko internal fields ka pata nahi hona chahiye, sirf ek opaque
// snapshot object hold karta hai.
// Chalao: node 2-memento.js

class Editor {
  #text = '';
  type(chars) { this.#text += chars; }
  get text() { return this.#text; }

  save() {
    return { text: this.#text }; // memento – apna hi state, khud banaya
  }
  restore(memento) {
    this.#text = memento.text; // sirf Editor khud jaanta hai kaise restore kare
  }
}

class History {
  #snapshots = [];
  push(memento) { this.#snapshots.push(memento); }
  pop() { return this.#snapshots.pop(); }
}
```

```

}

const editor = new Editor();
const history = new History();

editor.type('Hello');
history.push(editor.save()); // undo point save

editor.type(', world');
console.log('before undo:', editor.text);

editor.restore(history.pop()); // caller ko Editor ke internals ka pata hi nahi
console.log('after undo:', editor.text);

// Editor mein naya field (cursor position) add karo – sirf save()/restore()
// badlenge, caller ka History code CHHUE BINA rahega.

```

exercise.js — your turn

```

// 🐛 EXERCISE – game character ke state ko Memento se save/restore karo.
// Chalao: node exercise.js

// ----- naive version (isse refactor karna hai) -----
class Character {
  constructor() { this.hp = 100; this.level = 1; }
  takeDamage(dmg) { this.hp -= dmg; }
  levelUp() { this.level++; this.hp = 100; }
}

const hero = new Character();
const backup = { hp: hero.hp, level: hero.level }; // manual, error-prone
hero.takeDamage(80);
hero.levelUp();
console.log('current:', hero.hp, hero.level);
// "backup" restore karna hai to caller ko khud `hero.hp = backup.hp` likhna padega

// =====
// TODO 1: `Character` class mein `save()` method add karo jo current state
//         ka snapshot object return kare, aur `restore(memento)` method jo
//         usse wapas apply kare (2-memento.js jaisa).
//
// TODO 2: `SaveSlot` class (ya simple array) banao jo mementos store kare
//         – `save(memento)` aur `load()` methods.
//
// TODO 3 (bonus): Level up hone se pehle auto-save karo, dikhao ki
//                 `takeDamage` + `levelUp` ke baad bhi purane save pe restore ho
//                 sakta hai.
//
// Neeche apna solution likho, phir `node exercise.js` chala kar verify karo.
// =====

```

20. State

Problem: ek object ka behavior uske **current status** pe depend karta hai (order: placed/paid/shipped; traffic light: red/green/yellow), aur har method mein "abhi is status mein kya allowed hai" wala if/else duplicate hota hai. Invalid transitions rokna bhi mushkil hai.

Idea: har status ko ek object bana do jo khud jaanta ho — "is state mein kaunsa action allowed hai, aur uske baad next state kya hoga". Main object (Order/Light) sirf current state ko delegate karta hai.

Files — is order mein chalao

```
node 1-naive.js    # ❌ har method mein status if/else duplicate
node 2-state.js    # ✅ har status apna hi object, khud transitions decide karta hai
node exercise.js   # 🎯 tumhara turn
```

Principles yahan dikh rahe hain

1. **Behavior depends on state, delegate it** — Strategy jaisa hi mechanism (delegation), lekin state **khud badalta hai** (self-transition), Strategy mein caller badalta hai.

1. **Invalid transitions naturally blocked** — jo state jis action ko support nahi karta, uska handler khud "not allowed" bol deta hai.

When to skip

Agar sirf 2 states hain ya transitions kabhi badhenge nahi, simple boolean flag + if/else kaafi hai — State pattern tabhi kamaata hai jab ***kai states aur transitions*** ho jo complex ho rahe hon.

❌ 1-naive.js — the naive approach

```
// ❌ THE NAIVE VERSION — order status ek string hai, har jagah if/else se
// check hota hai, invalid transitions (delivered -> paid) rokta koi nahi.
// Chalao: node 1-naive.js
```

```

class Order {
  constructor() { this.status = 'placed'; }
  pay() {
    if (this.status === 'placed') { this.status = 'paid'; console.log('✅ paid'); }
    else console.log(`❌ cannot pay from status: ${this.status}`);
  }
  ship() {
    if (this.status === 'paid') { this.status = 'shipped'; console.log('✅ shipped'); }
    else console.log(`❌ cannot ship from status: ${this.status}`);
  }
  deliver() {
    if (this.status === 'shipped') { this.status = 'delivered'; console.log('✅ delivered'); }
    else console.log(`❌ cannot deliver from status: ${this.status}`);
  }
}

const order = new Order();
order.ship();    // invalid – abhi paid nahi hua
order.pay();
order.ship();
order.deliver();

// PROBLEM: har method mein "current status ke hisaab se kya allowed hai"
// wala if/else duplicate hai. Naya status (e.g. 'cancelled') add karo to
// HAR method dobara edit karna padega.

```

✅ 2-state.js — the pattern

```

// ✅ STATE PATTERN – har status ko ek object bana do jo khud decide kare
// "is status mein kya allowed hai". Order sirf current state ko delegate
// karta hai.
// Chalao: node 2-state.js

const placedState = {
  name: 'placed',
  pay: (order) => { console.log('✅ paid'); order.setState(paidState); },
  ship: () => console.log('❌ cannot ship – not paid yet'),
  deliver: () => console.log('❌ cannot deliver – not shipped yet'),
};

const paidState = {
  name: 'paid',
  pay: () => console.log('❌ already paid'),
  ship: (order) => { console.log('✅ shipped'); order.setState(shippedState); },
  deliver: () => console.log('❌ cannot deliver – not shipped yet'),
};

const shippedState = {
  name: 'shipped',
  pay: () => console.log('❌ already paid'),
  ship: () => console.log('❌ already shipped'),
  deliver: (order) => { console.log('✅ delivered'); order.setState(deliveredState); },
};

const deliveredState = {
  name: 'delivered',
  pay: () => console.log('❌ order already delivered'),

```

```

    ship: () => console.log('❌ order already delivered'),
    deliver: () => console.log('❌ already delivered'),
  };

  class Order {
    constructor() { this.state = placedState; }
    setState(state) { this.state = state; }
    pay() { this.state.pay(this); }
    ship() { this.state.ship(this); }
    deliver() { this.state.deliver(this); }
  }

  const order = new Order();
  order.ship(); // invalid - placedState.ship() khud "not paid" bolta hai
  order.pay();
  order.ship();
  order.deliver();

  // Naya status (cancelled) add karna = ek naya state object, Order class
  // ko CHHUE BINA.

```

exercise.js — your turn

```

// ⚙️ EXERCISE - traffic light (red -> green -> yellow -> red) ko State
// pattern se refactor karo.
// Chalao: node exercise.js

// ----- naive version (isse refactor karna hai) -----
class TrafficLight {
  constructor() { this.color = 'red'; }
  next() {
    if (this.color === 'red') this.color = 'green';
    else if (this.color === 'green') this.color = 'yellow';
    else if (this.color === 'yellow') this.color = 'red';
    console.log('now:', this.color);
  }
}

const light = new TrafficLight();
light.next(); light.next(); light.next();

// =====
// TODO 1: `redState`, `greenState`, `yellowState` objects banao, har ek
// mein `name` aur `next(light)` method (jo light.setState(...))
// call kare next color ke state pe - 2-state.js jaisa).
//
// TODO 2: `TrafficLight` class ko refactor karo - constructor mein
// `this.state = redState`, `next()` method `this.state.next(this)`
// call kare, `setState(state)` method ho.
//
// TODO 3 (bonus): socho - is design mein naya rule "yellow se seedha red,
// par sirf agar pedestrian button dabaya ho" add karna kitna easy
// hai naive version ke mukable?
//

```

```
// Neeche apna solution likho, phir `node exercise.js` chala kar verify karo.  
// =====
```

21. Template Method

Problem: kai variants mein ek SAME skeleton/sequence hota hai (load -> process -> export; welcome -> setup -> notify), lekin beech ka 1-2 step alag hota hai. Naive tareeke mein poora skeleton har variant mein duplicate hota hai.

Idea: skeleton ko ek base class mein **ek baar** fix karo (template method). Jo steps variant-specific hain, unhe abstract/overridable methods banao — subclasses sirf unhi ko implement karein, skeleton ko nahi chhuein.

Files — is order mein chalao

node 1-naive.js	# ❌	poora skeleton har report mein duplicate
node 2-template-method.js	# ✅	skeleton ek jagah, sirf steps override hote hain
node exercise.js	# 🔄	tumhara turn

Principles yahan dikh rahe hain

1. **Don't repeat the skeleton** — sequence/order ek hi jagah likha jaata hai.

1. **Hollywood Principle** — "don't call us, we'll call you": base class subclass ke methods ko sahi order mein khud call karta hai.

When to skip

Agar variants ke beech skeleton hi alag hai (sirf 1-2 steps common hain, poora flow nahi), Template Method zabardasti fit karoge to rigid ho jaayega — tab Strategy (poora behavior swap) ya plain functions behtar hain.

❌ 1-naive.js — the naive approach

```
// ❌ THE NAIVE VERSION — poora "load -> process -> export" skeleton har
// report type mein DUPLICATE hota hai, sirf beech ke steps alag hain.
// Chalao: node 1-naive.js
```

```
function generateCsvReport(data) {
  console.log('📁 loading data...');
```

```

const rows = data.map((d) => `${d.name},${d.value}`);
console.log('⚙️ processing...');
const processed = rows.join('\n');
console.log('📄 exporting...');
console.log('CSV OUTPUT:\n' + processed);
}

function generateJsonReport(data) {
  console.log('📁 loading data...'); // duplicate
  const processed = JSON.stringify(data);
  console.log('⚙️ processing...'); // duplicate
  console.log('📄 exporting...'); // duplicate
  console.log('JSON OUTPUT:\n' + processed);
}

generateCsvReport([{ name: 'A', value: 1 }]);
generateJsonReport([{ name: 'A', value: 1 }]);

// PROBLEM: "Load -> process -> export" ka skeleton har report function mein
// copy-paste hua hai. Skeleton mein koi step add/change karna ho (e.g. ek
// validation step) to HAR report function edit karna padega.

```

✅ 2-template-method.js — the pattern

```

// ✅ TEMPLATE METHOD — algorithm ka SKELETON base class mein ek baar likho,
// sirf jo steps subtype-specific hain unhe subclass override kare.
// Chalao: node 2-template-method.js

class ReportGenerator {
  // Template method — skeleton FIXED hai, subclass isse override nahi karti
  generate(data) {
    console.log('📁 loading data...');
    const processed = this.process(data); // yahi step subclass-specific hai
    console.log('⚙️ processing done');
    console.log('📄 exporting...');
    console.log(`${this.format()} OUTPUT:\n${processed}`);
  }
  process(data) { throw new Error('subclass must implement process()'); }
  format() { throw new Error('subclass must implement format()'); }
}

class CsvReport extends ReportGenerator {
  format() { return 'CSV'; }
  process(data) { return data.map((d) => `${d.name},${d.value}`).join('\n'); }
}

class JsonReport extends ReportGenerator {
  format() { return 'JSON'; }
  process(data) { return JSON.stringify(data); }
}

new CsvReport().generate([{ name: 'A', value: 1 }]);
new JsonReport().generate([{ name: 'A', value: 1 }]);

// Skeleton mein naya step (validation) add karna = ek jagah, generate() mein.

```

```
// Naya format (XML) add karna = bas process()+format() override karne wali
// ek nayi subclass, skeleton ko CHHUE BINA.
```

exercise.js — your turn

```
// 🐛 EXERCISE – do onboarding flows (free user vs paid user) ke duplicate
// skeleton ko Template Method se nikalo.
// Chalao: node exercise.js

// ----- naive version (isse refactor karna hai) -----
function onboardFreeUser(name) {
  console.log(`📧 welcome email to ${name}`);
  console.log('📁 free plan setup');
  console.log(`📢 notifying team: ${name} joined (free)`);
}

function onboardPaidUser(name) {
  console.log(`📧 welcome email to ${name}`); // duplicate
  console.log('📁 paid plan setup + billing');
  console.log(`📢 notifying team: ${name} joined (paid)`); // near-duplicate
}

onboardFreeUser('Asha');
onboardPaidUser('Rohit');

// =====
// TODO 1: `Onboarding` base class banao with template method `run(name)`
//         jo: welcome email (fixed) -> `this.setup(name)` (subclass-specific)
//         -> `this.notifyTeam(name)` (fixed, lekin plan naam ke liye
//         `this.planName()` ko call kare).
//
//
// TODO 2: `FreeOnboarding` aur `PaidOnboarding` subclasses banao jo
//         `setup(name)` aur `planName()` override karein.
//
//
// TODO 3 (bonus): naya `TrialOnboarding` add karo – dikhaao ki base class
//         `run()` ko CHHUE BINA kaam karta hai.
//
// Neeche apna solution likho, phir `node exercise.js` chala kar verify karo.
// =====
```

22. Visitor

Problem: classes ka ek fixed set hai (shapes, AST nodes), aur unpe naye **operations** add karte rehna hai (area, export, evaluate, print). Har naye operation ke liye har class ko edit karna padta hai — aur agar classes third-party se aayi ho to edit karna possible hi nahi.

Idea: naye operation ko class ke andar method ke bajaye ek alag "visitor" object mein rakho. Har class mein bas ek chhota, kabhi-na-badalne wala `accept(visitor)` method rehta hai jo sahi visitor-method ko call karta hai.

Files — is order mein chalao

```
node 1-naive.js      # ❌ naya operation = har shape class edit
node 2-visitor.js    # ✅ naya operation = naya visitor object, classes untouched
node exercise.js     # 🎯 tumhara turn
```

Principles yahan dikh rahe hain

1. **Add operations without modifying classes** — Open/Closed principle, operations ke perspective se.

1. **Double dispatch** — `shape.accept(visitor)` shape ka type decide karta hai, andar `visitor.visitX(this)` operation decide karta hai.

When to skip

Agar classes ka set frequently badalta hai (naya shape type baar-baar add hota hai) lekin operations stable hain, Visitor ulta pain hai — har naya shape ke liye HAR visitor ko edit karna padega. Visitor tab kamaata hai jab **classes stable hain, operations badalte rehte hain** (ulta scenario).

❌ 1-naive.js — the naive approach

```
// ❌ THE NAIVE VERSION — naya operation (e.g. "export to SVG") add karne ke
// liye HAR shape class ko edit karna padta hai.
// Chalao: node 1-naive.js
```

```

class Circle {
  constructor(r) { this.r = r; }
  area() { return Math.PI * this.r * this.r; }
  // naya operation chahiye? yahan naya method jodo... har shape mein.
}
class Square {
  constructor(s) { this.s = s; }
  area() { return this.s * this.s; }
}

const shapes = [new Circle(2), new Square(3)];
shapes.forEach((s) => console.log(s.constructor.name, 'area:', s.area().toFixed(2)));

// PROBLEM: agar kal "toSvg()" operation chahiye har shape ke liye, to Circle
// AUR Square dono classes ko edit karna padega. Jitne shapes utni jagah edit.
// Aur agar shapes third-party library se aayi hon (edit hi nahi kar sakte)?

```

✓ 2-visitor.js — the pattern

```

// ✓ VISITOR PATTERN – naye operations ko shape classes ke BAHAR, alag
// "visitor" objects mein rakho. Shapes sirf ek chhota `accept(visitor)`
// method rakhte hain – naya operation add karne ke liye unhe edit nahi
// karna padta.
// Chalao: node 2-visitor.js

class Circle {
  constructor(r) { this.r = r; }
  accept(visitor) { return visitor.visitCircle(this); } // fixed, kabhi nahi badalta
}
class Square {
  constructor(s) { this.s = s; }
  accept(visitor) { return visitor.visitSquare(this); }
}

// Operation #1: area calculation – ek visitor object
const areaVisitor = {
  visitCircle: (c) => Math.PI * c.r * c.r,
  visitSquare: (s) => s.s * s.s,
};

// Operation #2 (naya!): SVG export – shapes ko CHHUE BINA add hua
const svgVisitor = {
  visitCircle: (c) => `<circle r="${c.r}" />`,
  visitSquare: (s) => `<rect width="${s.s}" height="${s.s}" />`,
};

const shapes = [new Circle(2), new Square(3)];
shapes.forEach((s) => console.log('area:', s.accept(areaVisitor).toFixed(2)));
shapes.forEach((s) => console.log('svg:', s.accept(svgVisitor)));

// Naya operation (e.g. perimeter) add karna = ek naya visitor object,
// Circle/Square classes ko CHHUE BINA.

```

exercise.js — your turn

```
// 🐛 EXERCISE – simple expression tree (Number, Add nodes) ke Liye
// evaluate() aur print() operations ko Visitor se add karo, node classes ko
// chhue bina.
// Chalao: node exercise.js

// ----- given node classes (accept() already fixed hai, inhe mat badlo) -----
class NumberNode {
  constructor(value) { this.value = value; }
  accept(visitor) { return visitor.visitNumber(this); }
}
class AddNode {
  constructor(left, right) { this.left = left; this.right = right; }
  accept(visitor) { return visitor.visitAdd(this); }
}

// (2 + 3)
const tree = new AddNode(new NumberNode(2), new NumberNode(3));

// =====
// TODO 1: `evaluateVisitor` object banao with `visitNumber(node)` (return
//         node.value) aur `visitAdd(node)` (return node.left.accept(this) +
//         node.right.accept(this) – recursive!).
//
// TODO 2: `tree.accept(evaluateVisitor)` call karke result print karo
//         (chahiye: 5).
//
// TODO 3 (bonus): `printVisitor` banao jo expression ko string mein convert
//         kare (`"(2 + 3)"`), NumberNode/AddNode classes ko CHHUE BINA.
//
// Neeche apna solution likho, phir `node exercise.js` chala kar verify karo.
// =====
```

23. Interpreter

Problem: ek chhoti "language"/grammar hai (math expressions, boolean rules, filter queries) jise evaluate karna hai. String-parsing + if/else se karo to nested/composed expressions handle karna mushkil ho jaata hai.

Idea: grammar ke har rule (number, operator) ko ek chhoti class banao jismein `interpret()` method ho. Complex expressions in chhote nodes ko compose karke (tree ki tarah) banti hain — Composite pattern ka special use-case.

Files — is order mein chalao

```
node 1-naive.js      # ❌ string-parsing + if/else, sirf flat expressions
node 2-interpreter.js # ✅ chhoti composable classes, nested expressions natural
node exercise.js     # 🎯 tumhara turn
```

Principles yahan dikh rahe hain

1. **Grammar as objects** — har rule apna node-type, evaluation logic uske andar.
1. **Composable via recursion** — Composite jaisa hi idea, yahan operation ka naam `interpret()` hai.

When to skip

Agar grammar simple/fixed hai ya rarely change hoti hai, ek proper parser library (ya even `eval` -jaisa safe alternative) use karna behtar hai. Interpreter tab kamaata hai jab grammar ****chhoti ho aur custom rules frequently badalte/badhate**** hon.

❌ 1-naive.js — the naive approach

```
// ❌ THE NAIVE VERSION — simple math expressions ("3 + 4") ko ek hi function
// mein string-parsing + hardcoded if/else se evaluate kiya ja raha hai.
// Chalao: node 1-naive.js

function evaluate(expr) {
  const parts = expr.split(' ');
```

```

    if (parts.length === 3 && parts[1] === '+') return Number(parts[0]) + Number(parts[2]);
    if (parts.length === 3 && parts[1] === '-') return Number(parts[0]) - Number(parts[2]);
    throw new Error(`cannot evaluate: ${expr}`);
  }

```

```

console.log(evaluate('3 + 4'));
console.log(evaluate('10 - 6'));

```

*// PROBLEM: sirf DO numbers wale simple expressions handle hote hain.
 // Nested expressions ("3 + 4 - 2") ya naya operator add karna is function
 // ko bade if/else mein aur uljha dega – grammar aur evaluation mixed hain.*

✓ 2-interpreter.js — the pattern

*// ✓ INTERPRETER PATTERN – grammar ke har "rule" ko ek chhoti class banao
 // jismein `interpret()` method ho. Complex expressions in chhote nodes ko
 // COMPOSE karke banti hain (jaisa Composite pattern).
 // Chalao: node 2-interpreter.js*

```

class NumberExpr {
  constructor(value) { this.value = value; }
  interpret() { return this.value; }
}
class AddExpr {
  constructor(left, right) { this.left = left; this.right = right; }
  interpret() { return this.left.interpret() + this.right.interpret(); }
}
class SubtractExpr {
  constructor(left, right) { this.left = left; this.right = right; }
  interpret() { return this.left.interpret() - this.right.interpret(); }
}

```

// "3 + 4 - 2" ko tree se build karo – har node grammar ka ek rule hai

```

const expr = new SubtractExpr(
  new AddExpr(new NumberExpr(3), new NumberExpr(4)),
  new NumberExpr(2),
);
console.log('3 + 4 - 2 =', expr.interpret());

```

*// Naya operator (multiply) add karna = ek nayi chhoti class, existing
 // classes ko CHHUE BINA. Nested expressions naturally kaam karte hain
 // kyunki har node bas .interpret() call karta hai apne children pe.*

🔧 exercise.js — your turn

*// 🔧 EXERCISE – feature-flag rules ("isAdmin AND isBeta") ko Interpreter
 // pattern se evaluate-able banao.
 // Chalao: node exercise.js*

```

const context = { isAdmin: true, isBeta: false, isPremium: true };

```

// ----- naive version (isse refactor karna hai) -----

```
function evalRule(rule, ctx) {
  // sirf yeh ek hardcoded case handle karta hai:
  if (rule === 'isAdmin AND isBeta') return ctx.isAdmin && ctx.isBeta;
  throw new Error(`cannot evaluate: ${rule}`);
}
console.log(evalRule('isAdmin AND isBeta', context));

// =====
// TODO 1: `VarExpr` class banao (constructor: name) with `interpret(ctx)`
//         jo `ctx[this.name]` return kare.
//
// TODO 2: `AndExpr` aur `OrExpr` classes banao (constructor: left, right)
//         with `interpret(ctx)` jo `left.interpret(ctx) && right.interpret(ctx)`
//         (ya `||`) return karein.
//
// TODO 3 (bonus): `NotExpr` class banao (constructor: expr) with
//                 `interpret(ctx)` jo `!expr.interpret(ctx)` return kare. Phir
//                 "isAdmin AND NOT isBeta" jaisa tree bana kar evaluate karo.
//
// Neeche apna solution likho, phir `node exercise.js` chala kar verify karo.
// =====
```